JOHNS HOPKINS
APPLIED PHYSICS LABORATORY

Flash Orin with RHEL 9.4 + JetPack R36.4.0

Shunguang Wu
AOS/QPJ
14 Aug 2026

Ref:

1 https://developers.redhat.com/learning/learn:rhel:install-red-hat-device-edge-nvidia-jetson-orin-and-igx-orin/resource/resources:validate-jetson-gpu-support
2. chatgpt

UNCLASSIFIED

Outline

- Part 1 Flash Orin with JetPack R36.4.0
- Part 2 Install RHEL 9.4 ISO
- Part 3 Set up NVIDIA Kernel Module and Utilities
- Part 4 Tests and Evaluations

A. Install on eMMC

1. Flash NVIDIA QSPI firmware
↓
2. Boot from RHEL installer USB
↓
3. Install RHEL on eMMC
↓
4. Remove USB and reboot
↓
5. Verify RHEL is running from eMMC
↓
6. Install NVIDIA/JetPack packages
↓
7. Reboot and verify GPU + GUI work
↓
8. Encrypt the NVMe with LUKS
↓
9. Create filesystem on encrypted NVMe
↓
10. Configure /etc/crypttab + /etc/fstab
↓
11. Test and Evaluate NVIDIA native CUDA related applications
↓
12. Build & test ffmpeg with NVMPI

B. Install on NVME

1. Flash NVIDIA QSPI firmware
↓
2. Boot from RHEL installer USB
↓
3. Install RHEL on NVME SSD
↓
4. Remove USB and reboot
↓
5. Verify RHEL is running from NVME
↓
6. Install NVIDIA/JetPack packages
↓
7. Reboot and verify GPU + GUI work
↓
8. Test and Evaluate NVIDIA native CUDA related applications
↓
9. Build & test ffmpeg with NVMPI

Part 1: Flashing Orin with R36.4.0



USB connection



Host: RHEL 8.10
(192.168.7.98)

Target: Orin

Download JetPack firmware and dtc rpm files

- Download NVIDIA Orin AGX firmware and dtc rpm files
  - ✓ dtc-1.6.0-1.el8.x86_64.rpm
  - ✓ dtc-1.6.0-1.el8.x86_64.rpm.sha256
  - ✓ Jetson_Linux_R36.4.0_aarch64.tbz2
  - ✓ Jetson_Linux_R36.4.0_aarch64.tbz2.sha256

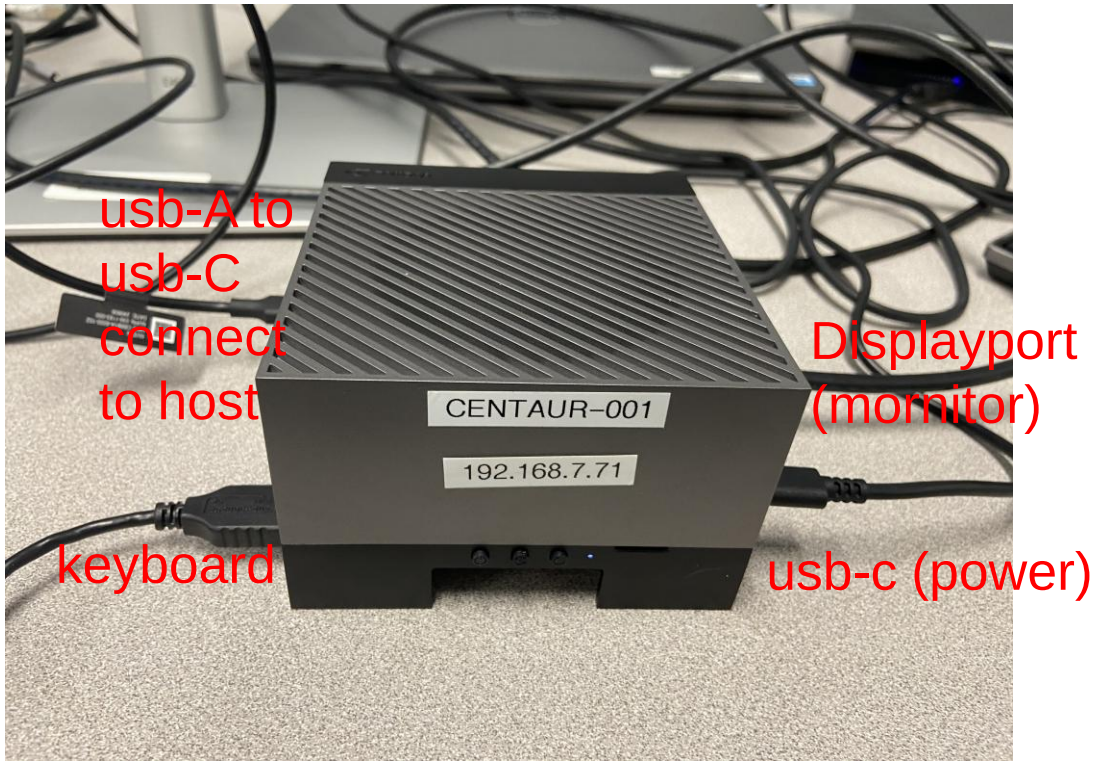
  Note:
  - ✓ Any packages must be downloaded from Nvidia and RedHat Linux official web
  - ✓ These files are available in bfpgw1: /usr/people/wus1**`/flash_orin_rhel`**
  - ✓ Or you can download them from https://developer.nvidia.com to your APLNIS machine, perform an anti-virus scan.

Orin connection ports

- Hardware connection
  - ✓ USB-A to USB-C cable
  - ✓ DisplayPort cable to monitor
  - ✓ Keyboard and mouse connect to Orin
  - ✓ Power cable: USB-C or DC jack

- Warm up commands on host machine

  $ su -
  $ cat /etc/os-release
  $ lsb_release -a
  $ lsusb



Flashing Orin

- On Host
  - copy download **tbz2** and dtc files to host machine at: /opt/export
  - extract Jetson_Linux_R36.4.0_aarch64.tbz2 file to: /opt/export/R36.4.0

    $su -

    $cd /opt/export

    $tar xf jetson_linux_r36.4.0_aarch64.tbz2 -C ./R36.4.0

  - The extracted results will be in /opt/export/R36.4.0/Linux_for_Tegra
- Set Orin AGX in “recovery” mode
  - Take off the power cable, then press and hold the middle button meanwhile plug in continuously hold for a few seconds. (see “Preparation and setup” from the reference)
- On Host terminal window do:
  - go to /opt/export/
  - install the **dtc** package on your Minotaur laptop

    -- not part of the normal Minotaur ISO,

    -- only need to do it once on the host machine if the dtc is not installed on that machine

    $cd /opt/export

    $dnf –disablerepo=‘*’ install ./dtc-1.6.0-1.el8.x86_64.rpm
  - go to /opt/export/R36.4.0/Linux_for_Tegra

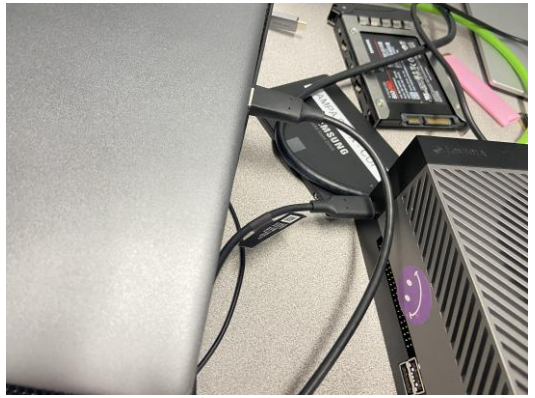
    (1) ensure the device is in recovery mode, then run:

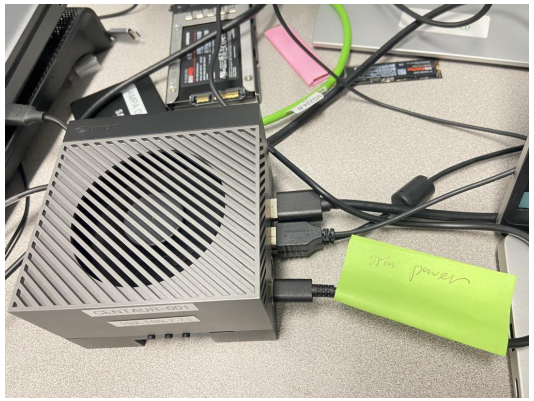
    $lsusb | grep NVIDIA

    (2)flash by:

    $./flash.sh p3737-0000-p3701-0000-qspi external

    (Note: if the deice is not Orin AGX Developer kit, you need to change p3737-* parameter to match your device)
  - It will take a few minutes to complete, at the end you will see the right figure.
  - Power off Orin





```
2. bfpgw1
[ 423.2071 ] Writing partition secondary_gpt_backup with gpt_backup_secondary_3_0.bin [ 16896 bytes ]
[ 424.0518 ] [................................................] 100%
[ 424.0566 ] Writing partition B_VER with qspi_bootblob_ver.txt [ 109 bytes ]
[ 424.2709 ] [................................................] 100%
[ 424.2742 ] Writing partition A_VER with qspi_bootblob_ver.txt [ 109 bytes ]
[ 424.2896 ] [................................................] 100%
[ 424.3103 ] tegradevflash_v2 --write BCT br_bct_BR.bct
[ 424.3114 ] Bootloader version 01.00.0000
[ 424.3358 ] Writing partition BCT with br_bct_BR.bct [ 8192 bytes ]
[ 424.3360 ] [................................................] 100%
[ 430.9879 ] tegradevflash_v2 --write BCT-boot-chain_backup bct_backup.img
[ 430.9887 ] Bootloader version 01.00.0000
[ 431.0128 ] Writing partition BCT-boot-chain_backup with bct_backup.img [ 32768 bytes ]
[ 431.0130 ] [................................................] 100%
[ 431.4310 ] tegradevflash_v2 --write A_MB1_BCT mb1_cold_boot_bct_MB1_sigheader.bct.encrypt
[ 431.4317 ] Bootloader version 01.00.0000
[ 431.4549 ] Writing partition A_MB1_BCT with mb1_cold_boot_bct_MB1_sigheader.bct.encrypt [ 18688 bytes ]
[ 431.4552 ] [................................................] 100%
[ 431.7075 ] tegradevflash_v2 --write B_MB1_BCT mb1_cold_boot_bct_MB1_sigheader.bct.encrypt
[ 431.7084 ] Bootloader version 01.00.0000
[ 431.7320 ] Writing partition B_MB1_BCT with mb1_cold_boot_bct_MB1_sigheader.bct.encrypt [ 18688 bytes ]
[ 431.7323 ] [................................................] 100%
[ 431.9852 ] tegradevflash_v2 --write A_MEM_BCT mem_coldboot_sigheader.bct.encrypt
[ 431.9861 ] Bootloader version 01.00.0000
[ 432.0100 ] Writing partition A_MEM_BCT with mem_coldboot_sigheader.bct.encrypt [ 243712 bytes ]
[ 432.0102 ] [................................................] 100%
[ 435.0515 ] tegradevflash_v2 --write B_MEM_BCT mem_coldboot_sigheader.bct.encrypt
[ 435.0525 ] Bootloader version 01.00.0000
[ 435.0767 ] Writing partition B_MEM_BCT with mem_coldboot_sigheader.bct.encrypt [ 243712 bytes ]
[ 435.0769 ] [................................................] 100%
[ 438.1169 ] Flashing completed

[ 438.1170 ] Coldbooting the device
[ 438.1183 ] tegrarcm_v2 --chip 0x23 0 --ismb2
[ 438.1189 ] MB2 version 01.00.0000
[ 438.1439 ] Coldbooting the device
[ 438.1452 ] tegrarcm_v2 --chip 0x23 0 --reboot coldboot
[ 438.1461 ] MB2 version 01.00.0000
*** The target generic has been flashed successfully. ***
Make the target filesystem available to the device and reset the board to boot from external external.

[root@mp Linux_for_Tegra]#
```

Part 2: Install RHEL9.4 ISO

On Host: Prepare a USB Drive for Boot

0. check iso file exist: /opt/export/rhel-9.4-aarch64-dvd.iso
1. Find a USB drive which is larger than 16GB, then do the following as root:
2. Plug the usb into host machine

```
# Verify that /dev/sda really is your USB stick
$lsblk -o NAME,SIZE,MODEL,TRAN,MOUNTPOINTS
NAME          SIZE MODEL           MOUNTPOINT
loop0         8.7G                 /mnt/rhel
loop1        12.4G                 /mnt/rhel_9.8_arm64
sdb         114.6G SanDisk 3.2Gen1
└─sdb1      114.6G                 /mnt/usb

# Unmount any mounted USB partitions
$umount /mnt/usb
$mount /dev/sdb1 /mnt/usb

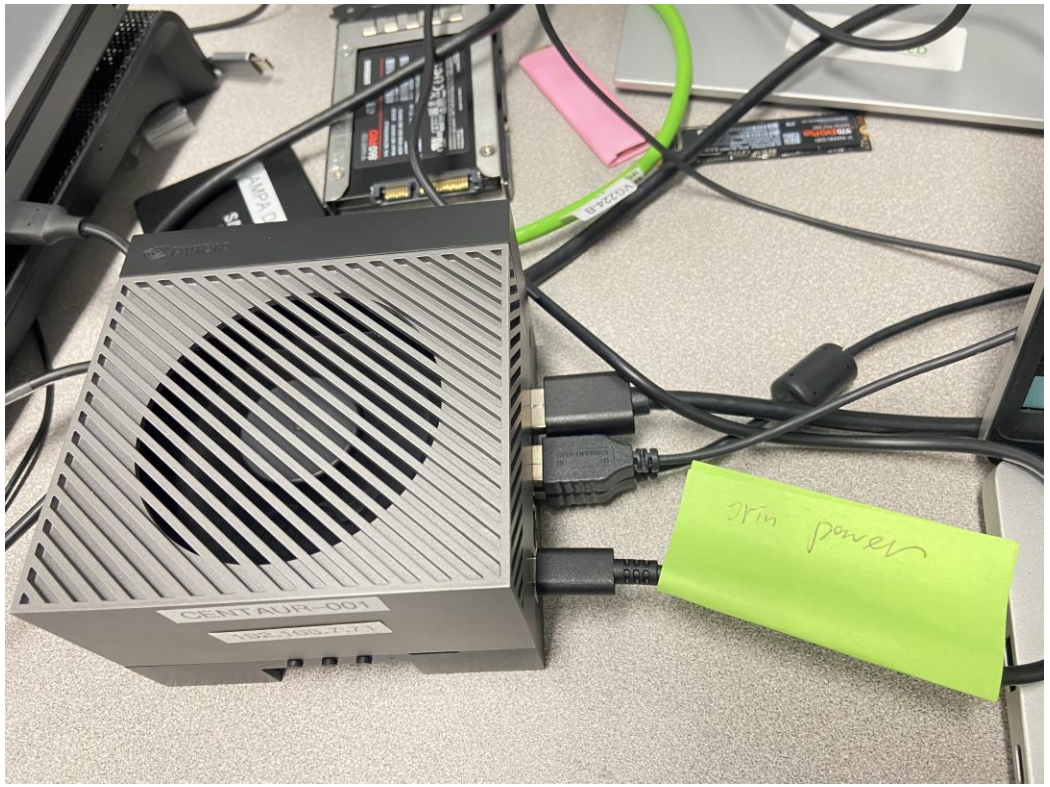
# Write the ISO to the whole USB device
$dd if=/opt/export/rhel-9.4-aarch64-dvd.iso \
    of=/dev/sdb \
    bs=4M \
    status=progress \
    oflag=sync

# unmount usb
$umount /mnt/usb
```

3. Pullout the usb drive from host machine

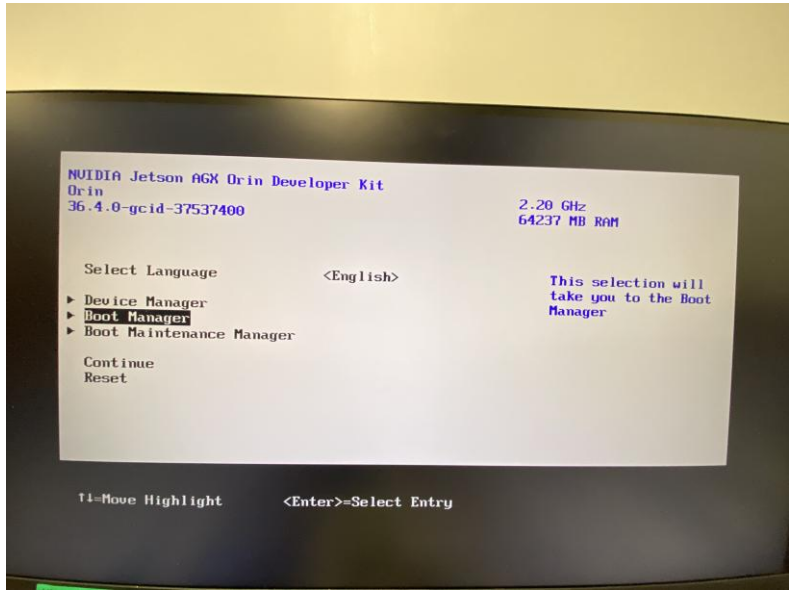
On Target

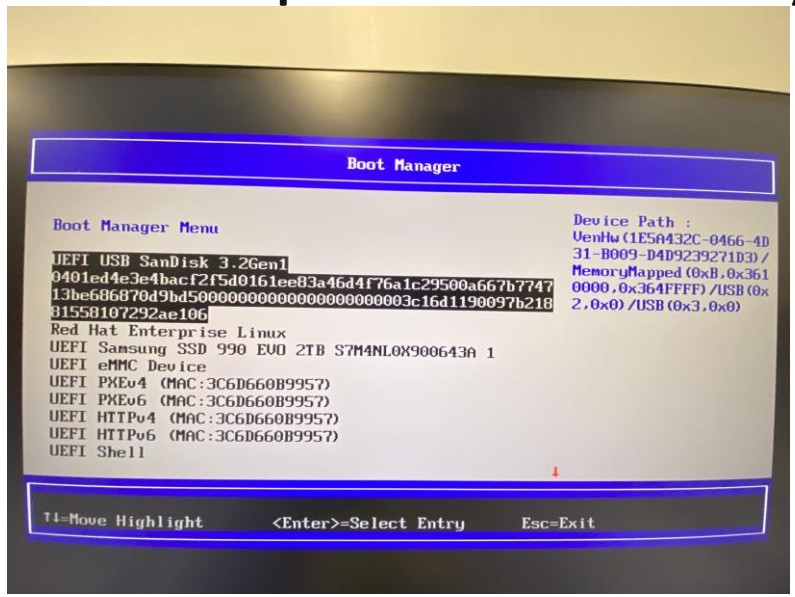
0. Power off Orin
1. Plug the USB boot drive into Orin
2. Power on Orin and the frequently press esc key
3. Select items in order (see next slides)

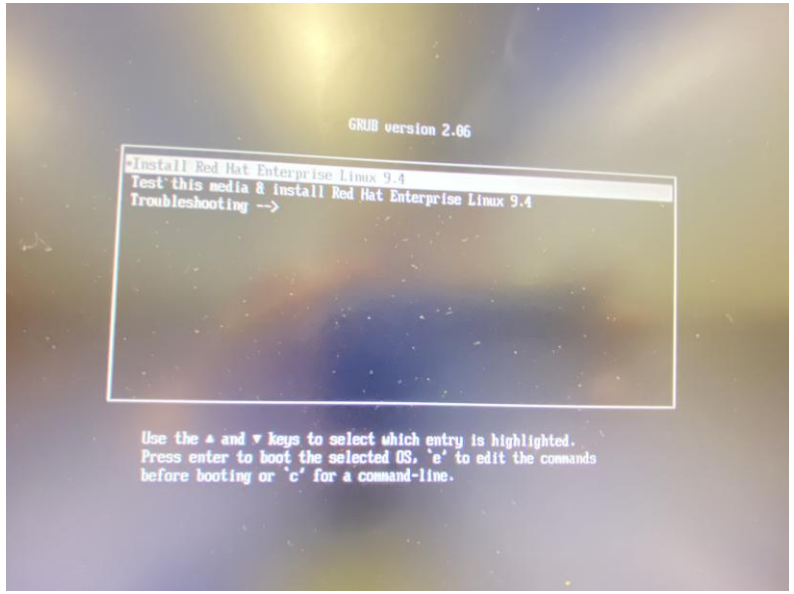


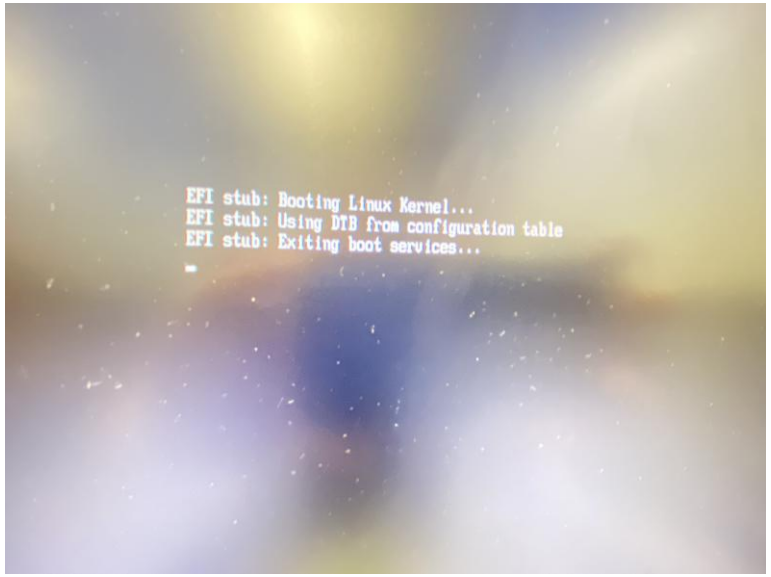
Orin connected with keyborad, mouse and monitor

Set up Red Hat Enterprise Linux (use tab and up and down keys)

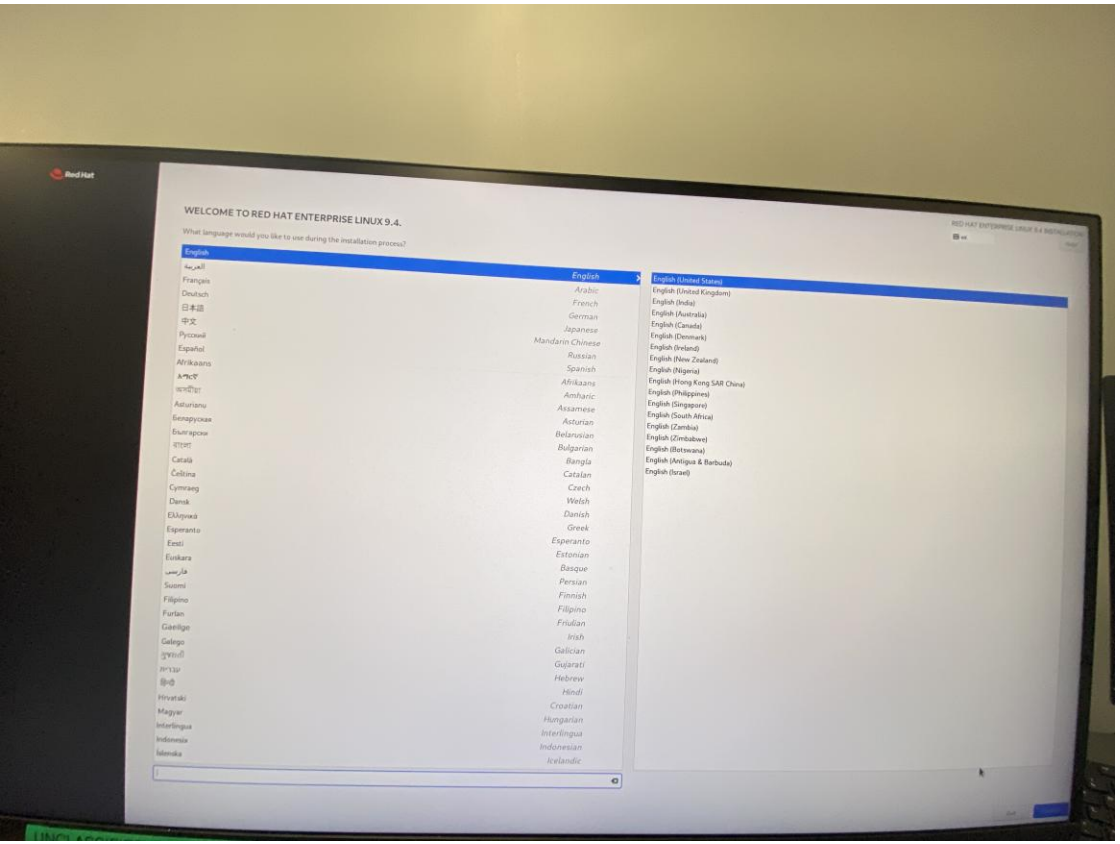




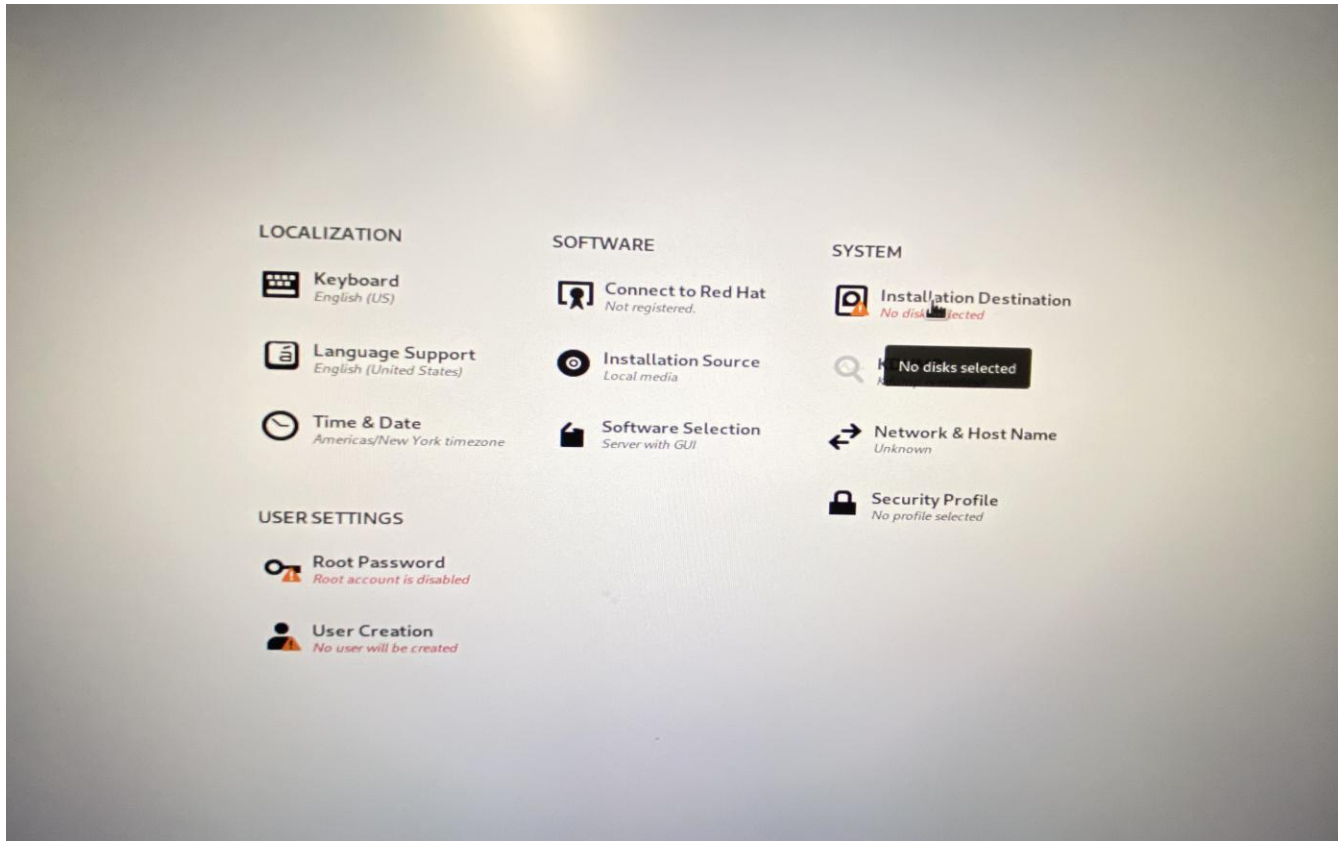




Select language and Installation destination

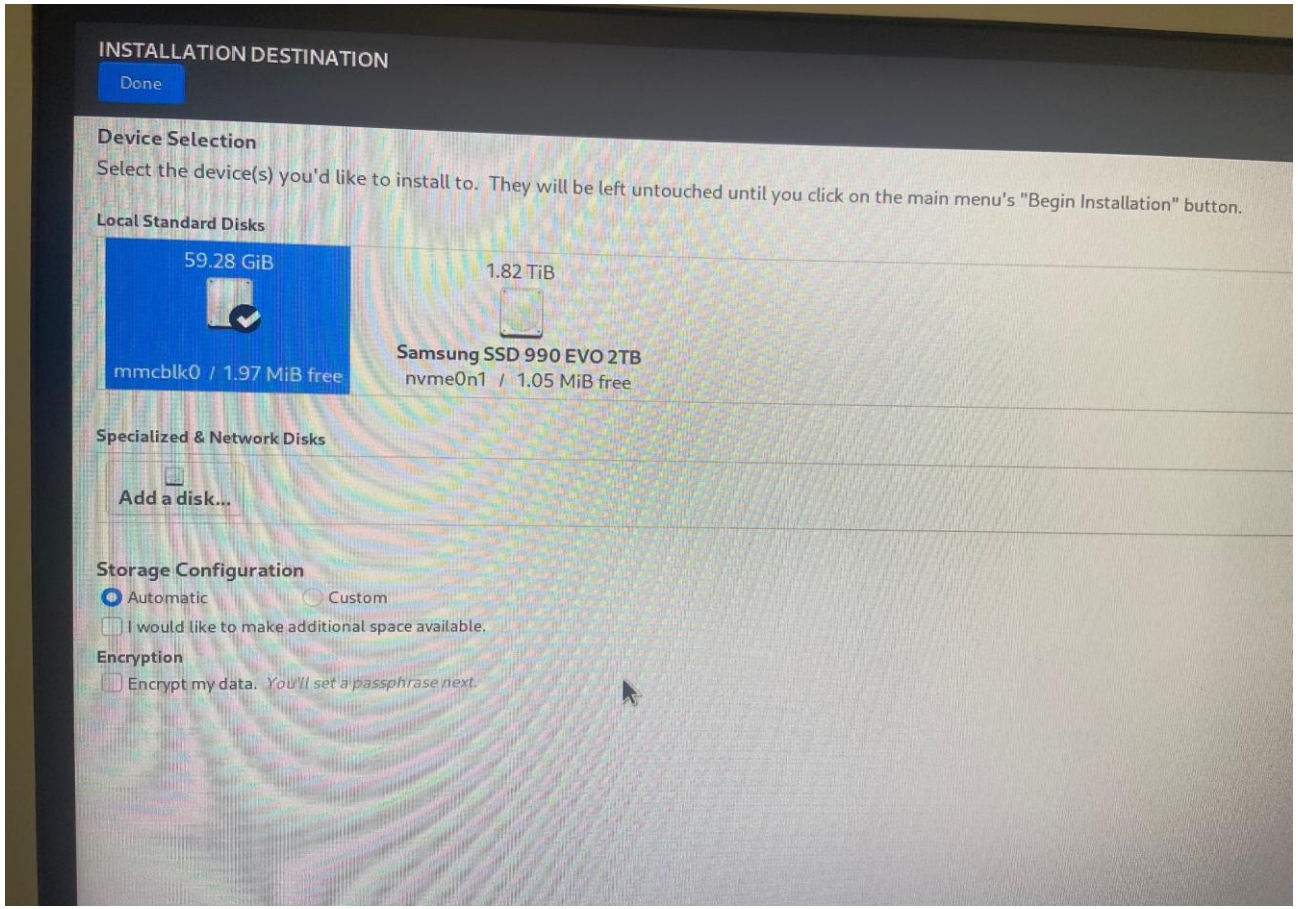


Choose language

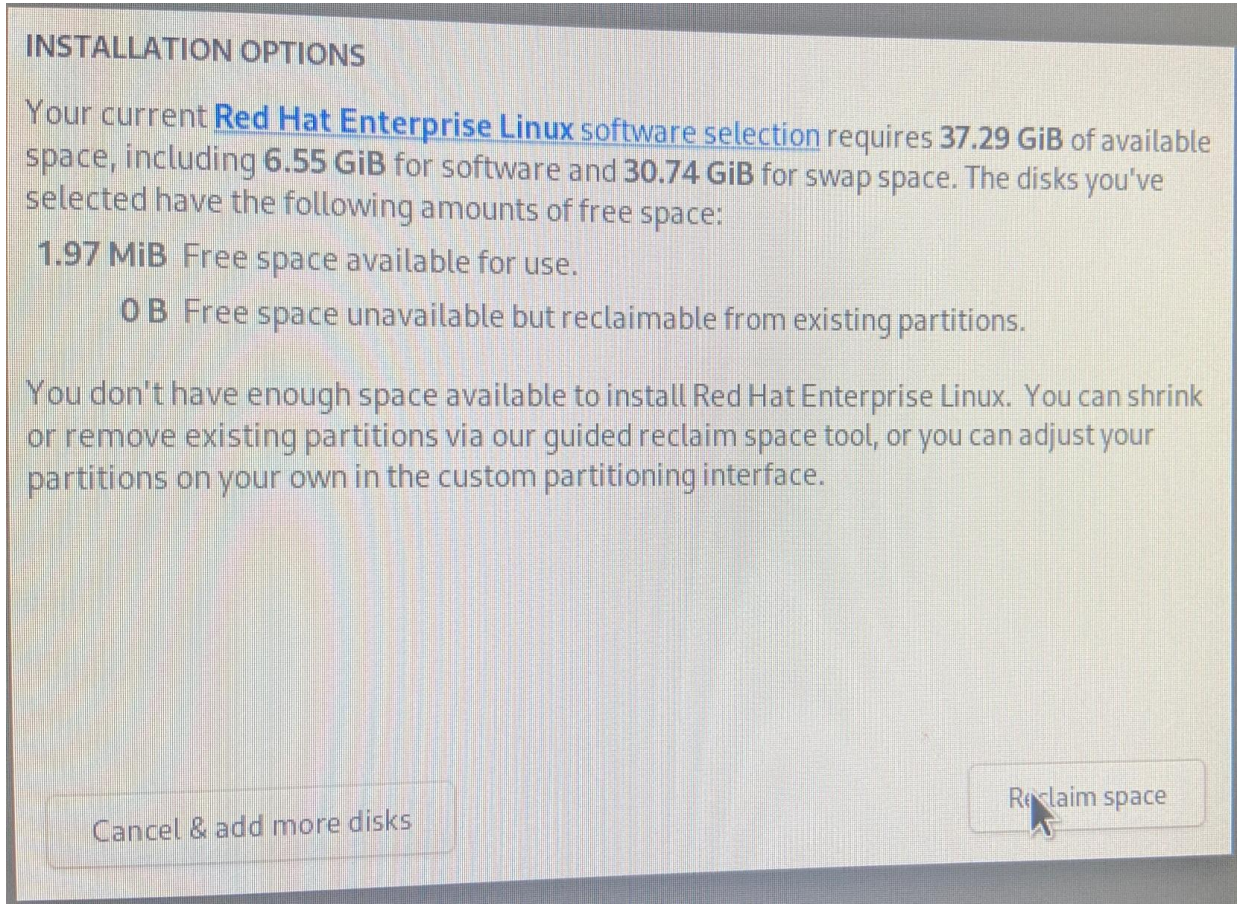


Choose where to install OS

Approach A: Install on eMMC

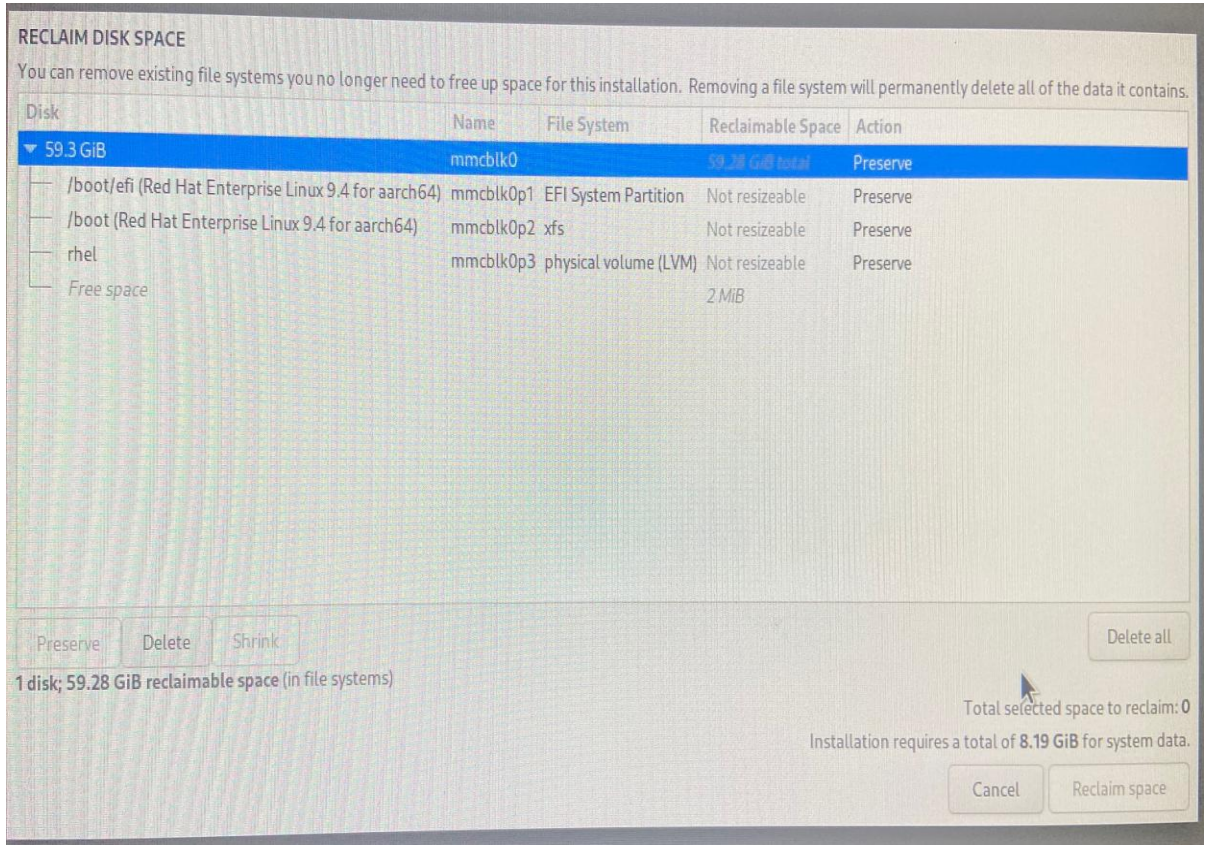


We want to install on NVME

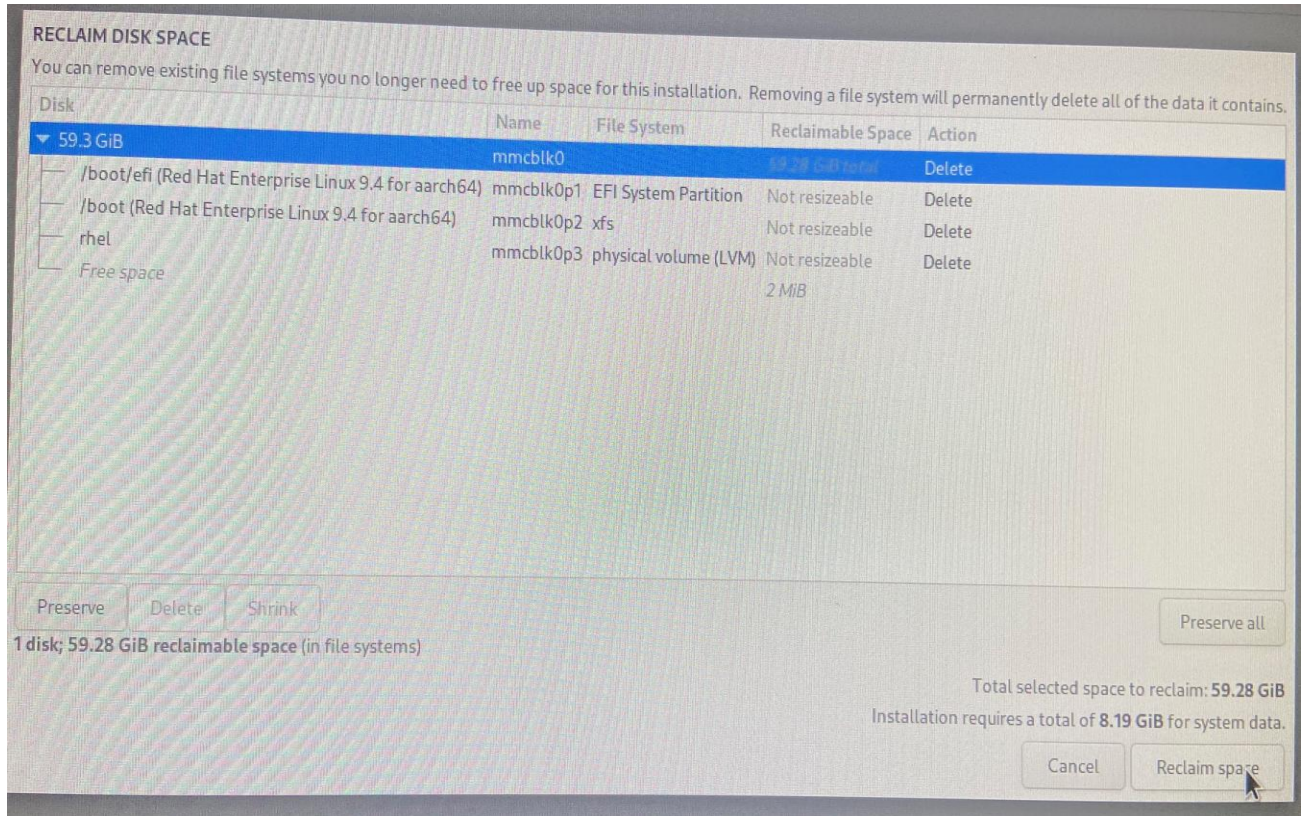


Reclaim space

Approach A: Install on eMMC

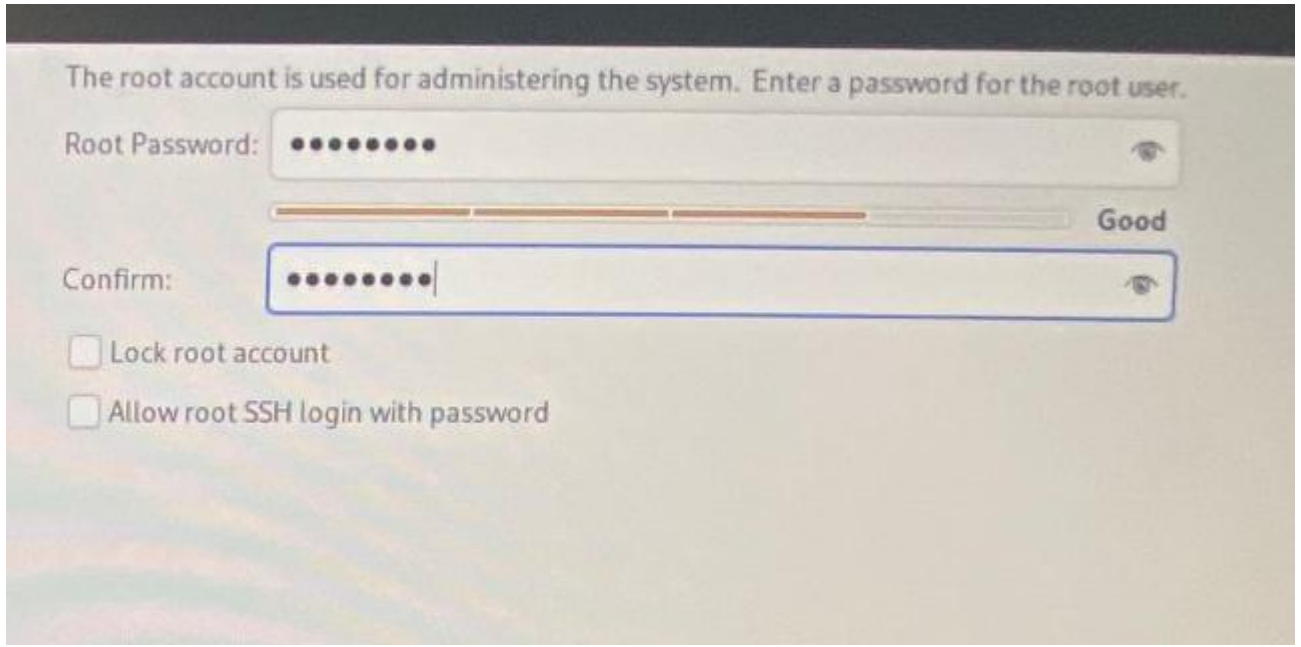


Select “Delete all”

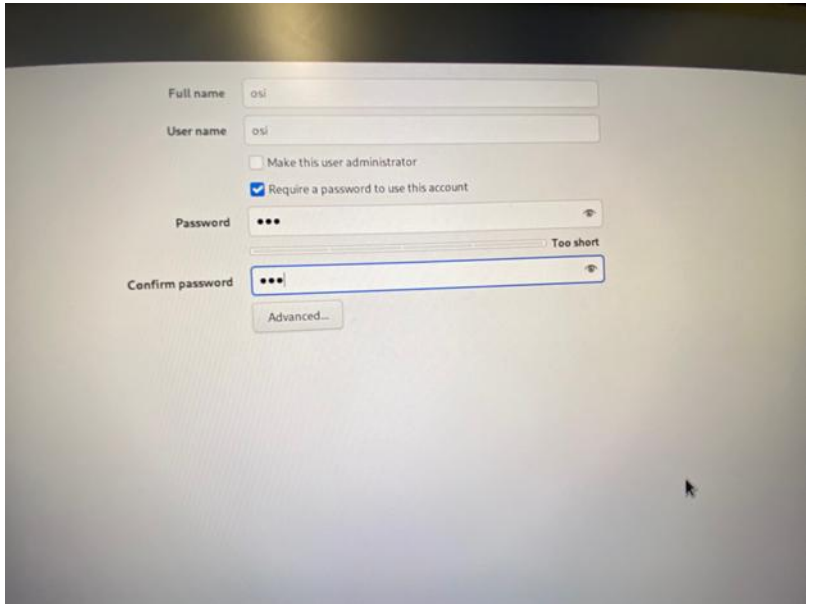


Select “Reclaim space”

Approach A: user settings

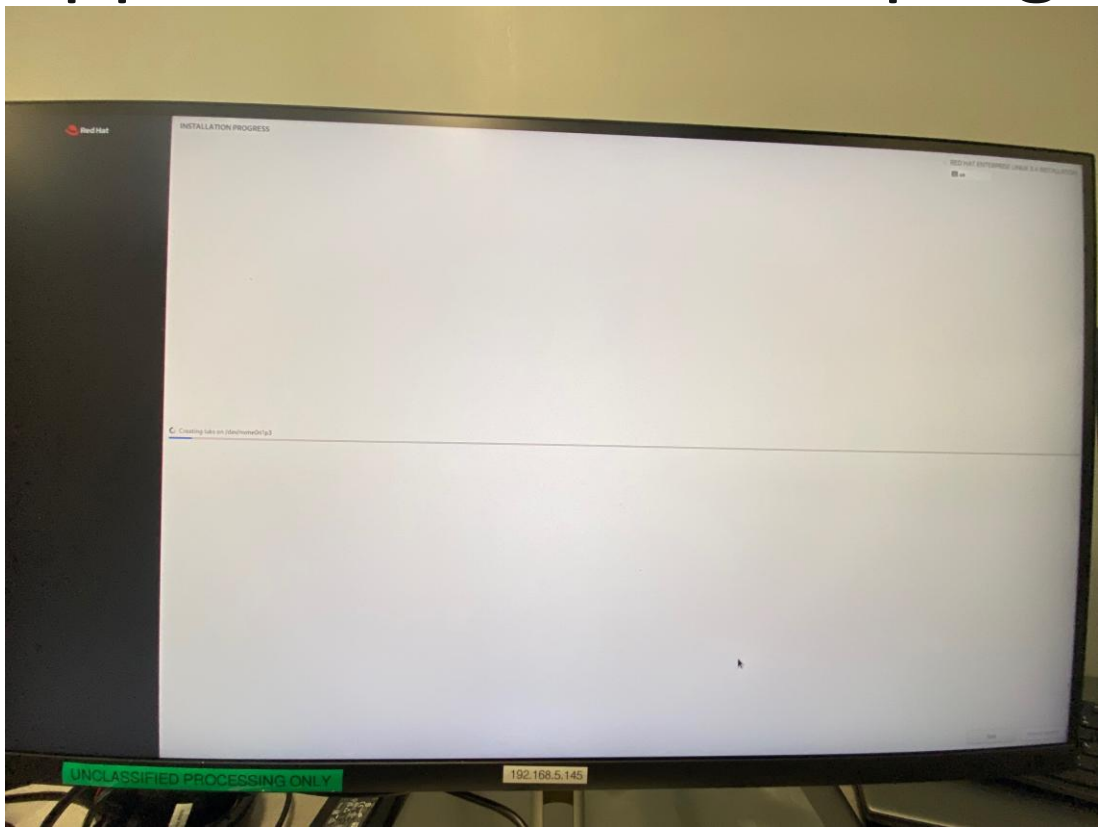


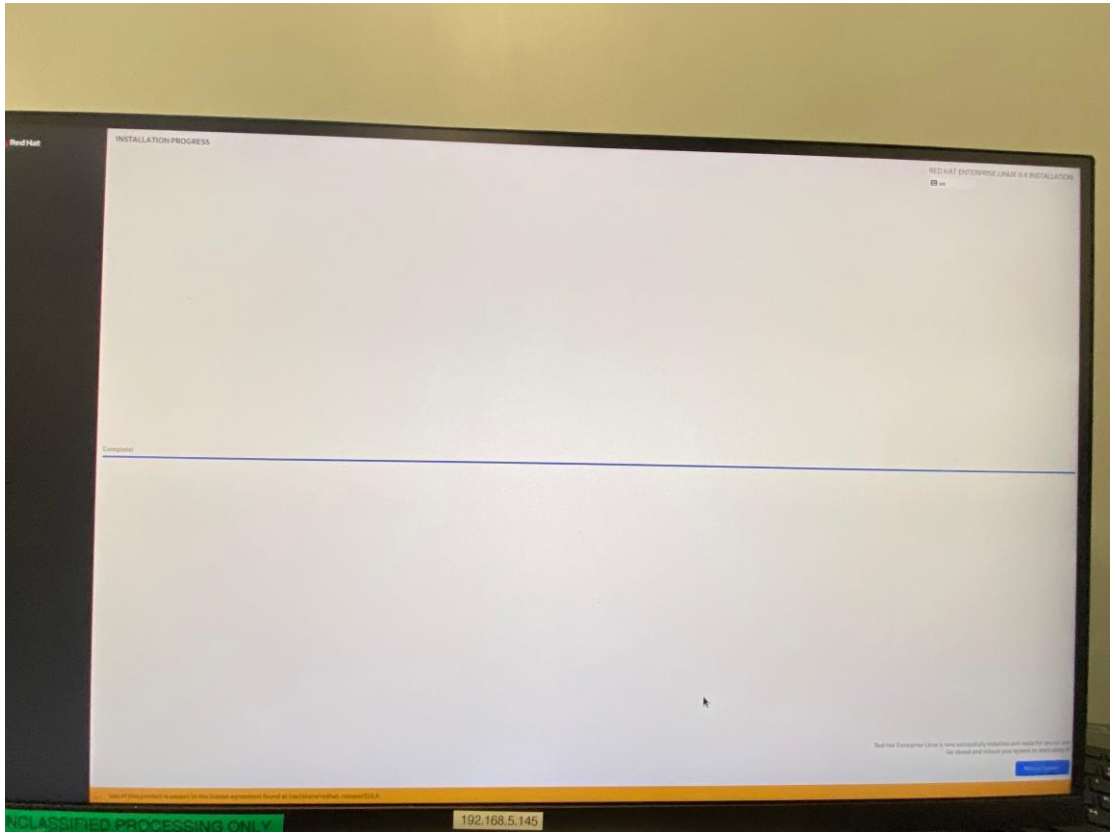
Input the password of root



set user osi and its password
since it’s too short, you need to click “Done” twice

Approach A: Install on progress and “reboot” at the end



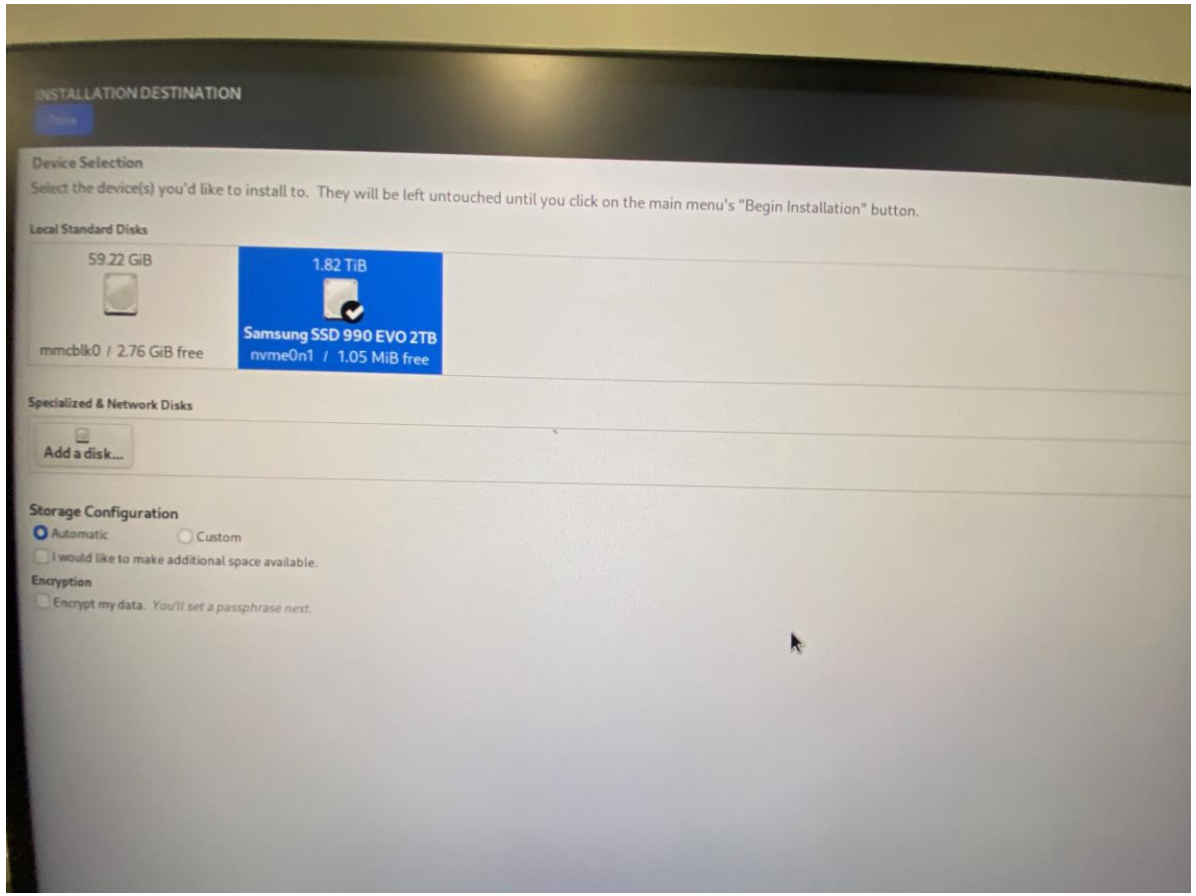


It is install iso, just wait

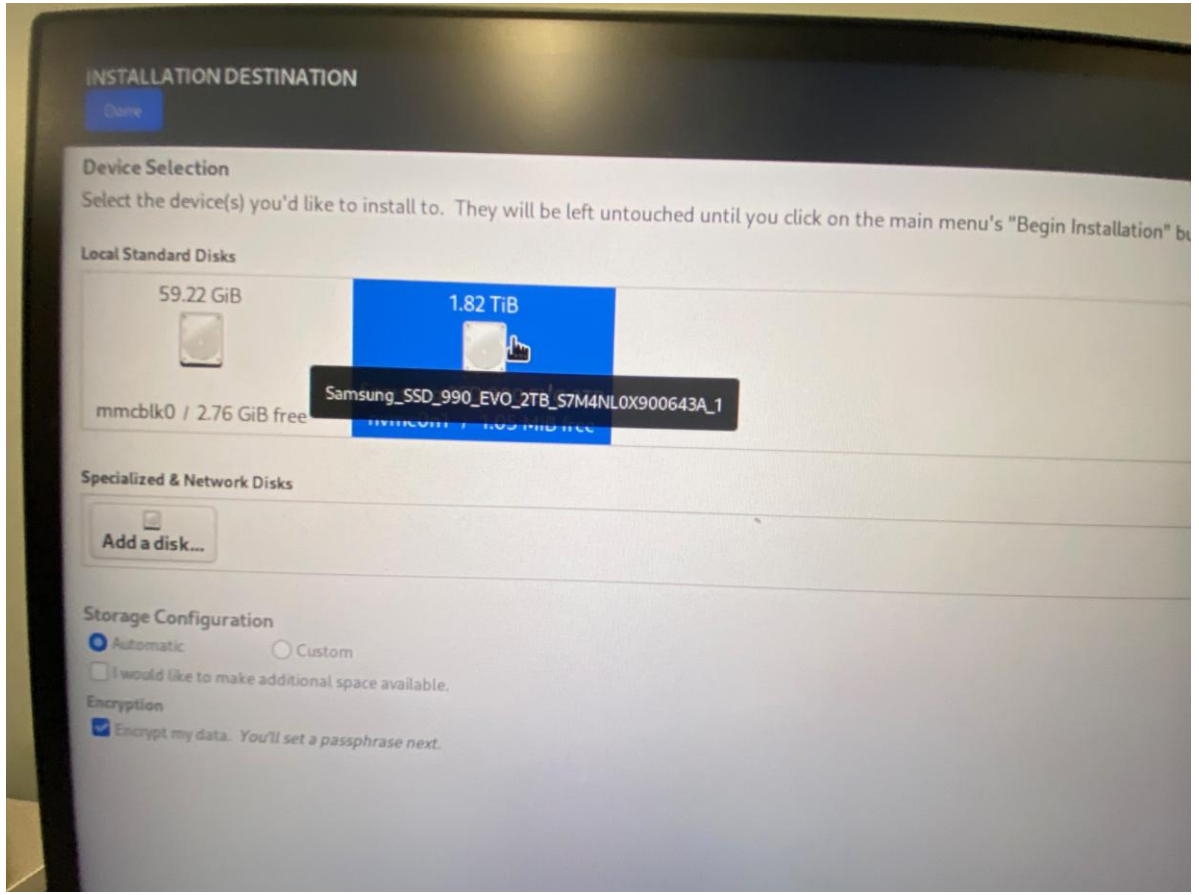
at the end “reboot” button will be active

click “reboot”, then power off the system from desktop interface
At this stage the approach A is done!

Approach B: Install on NVME

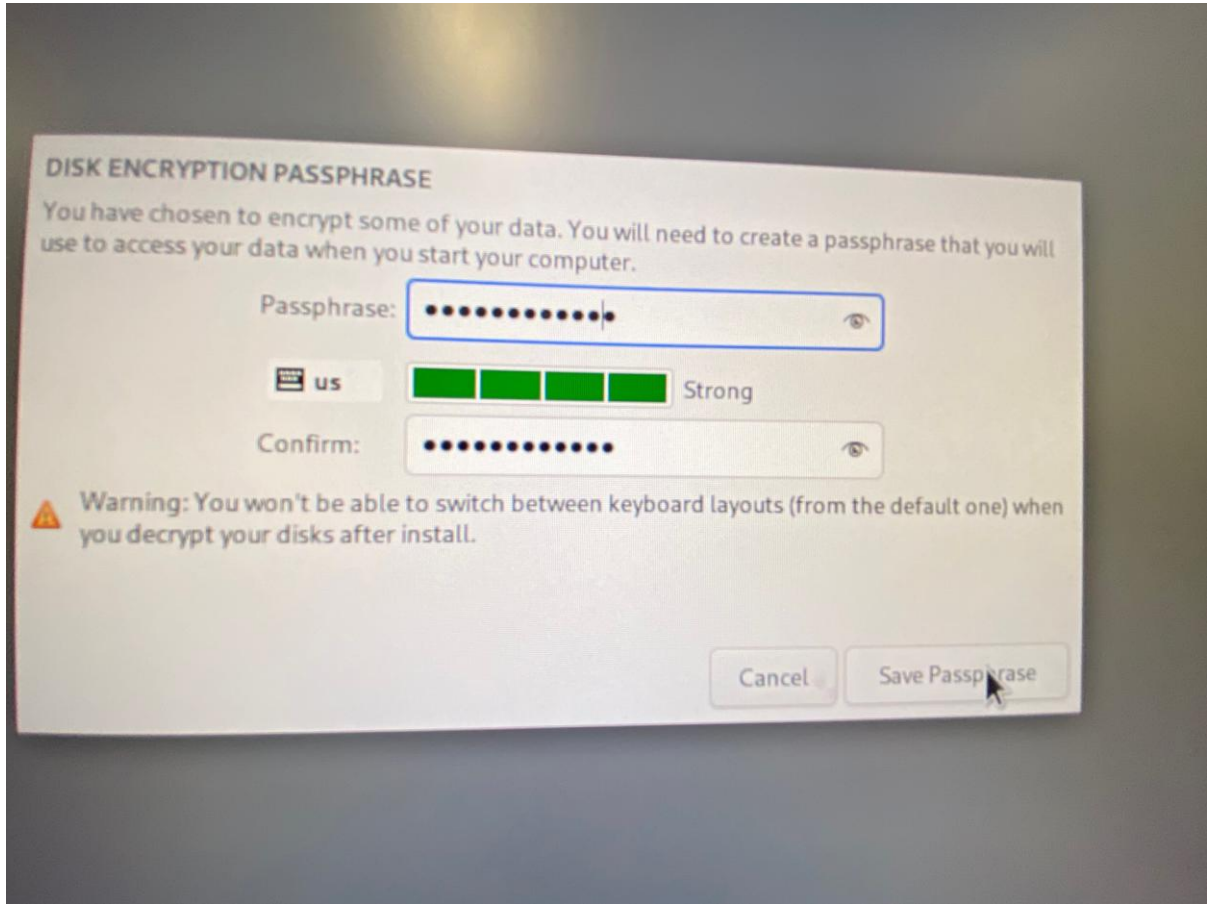


We want to install on NVME

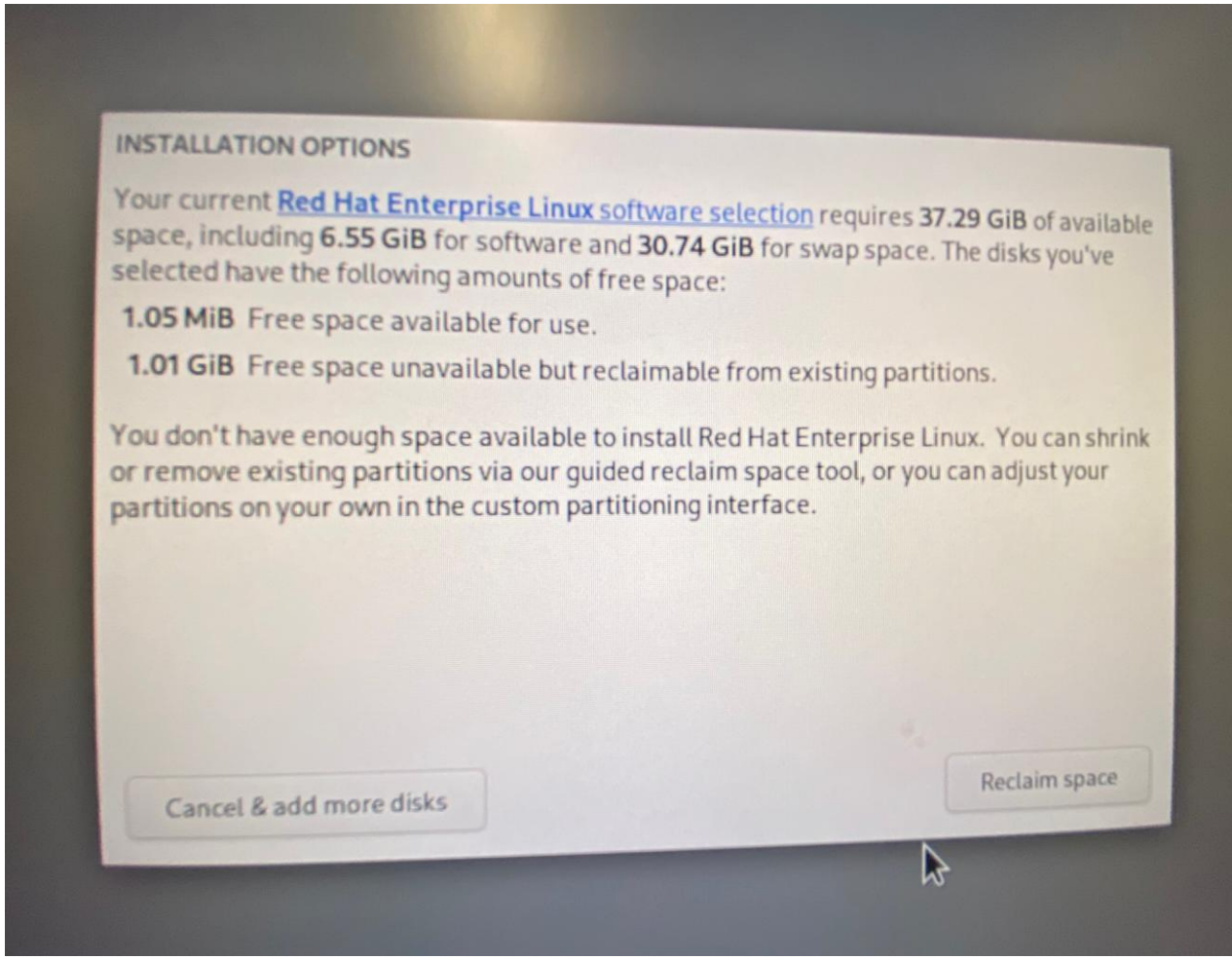


Select encryption

Approach B: Input Password of Disk Encryption

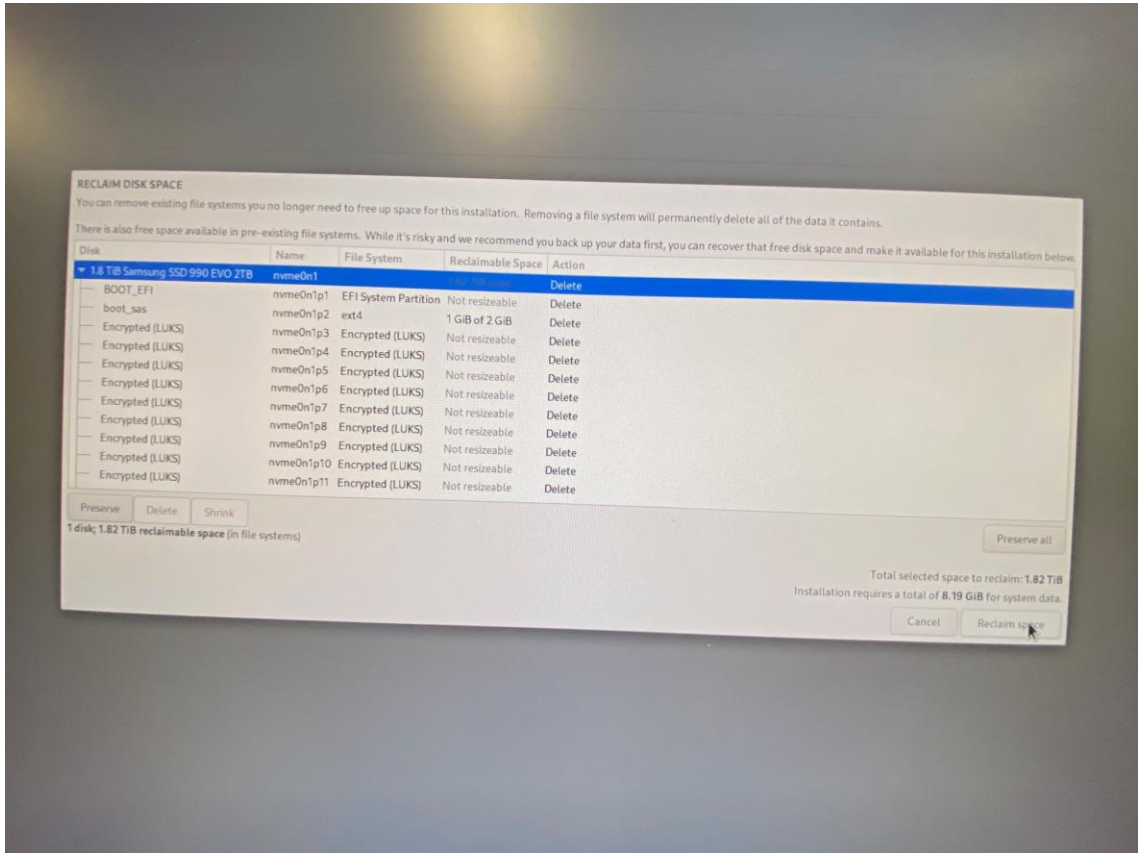


Input our favorite password to encrypt the hard drive

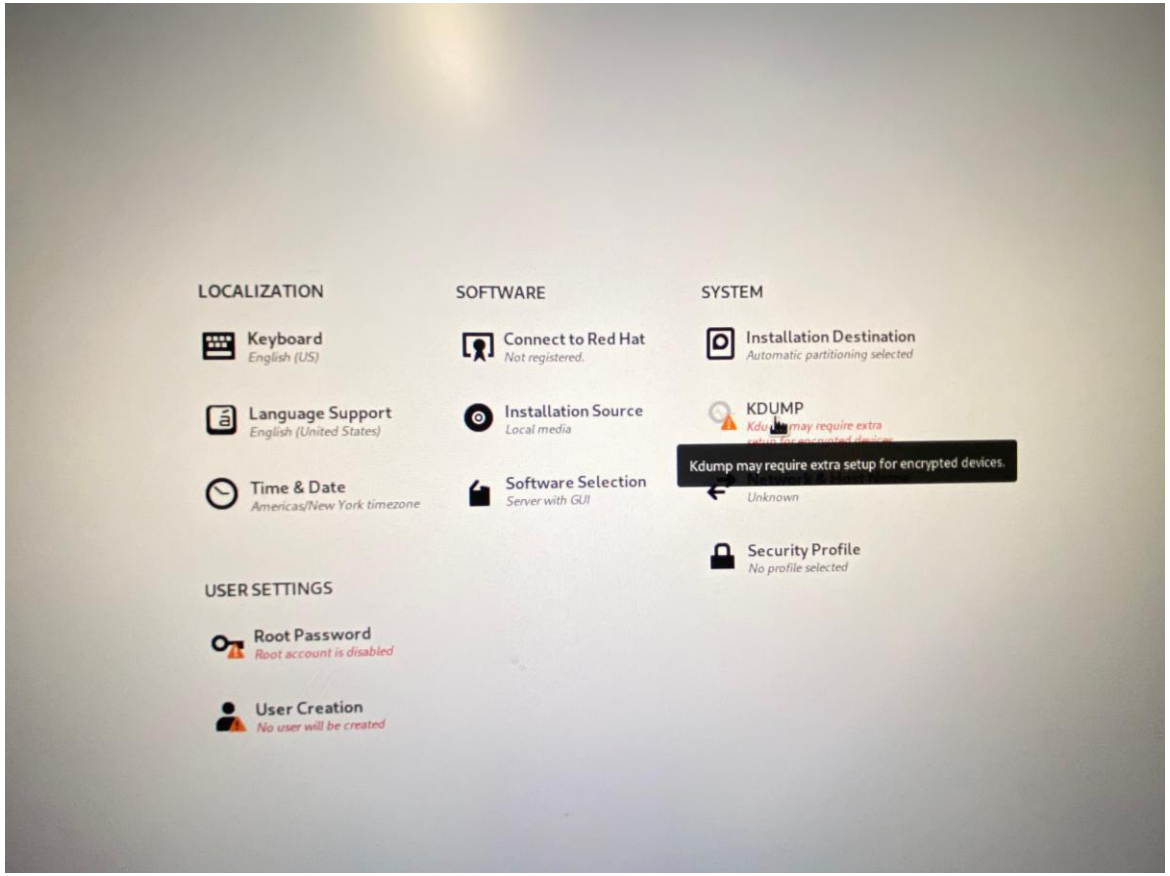


Click Reclaim space

Approach B: reclaim space

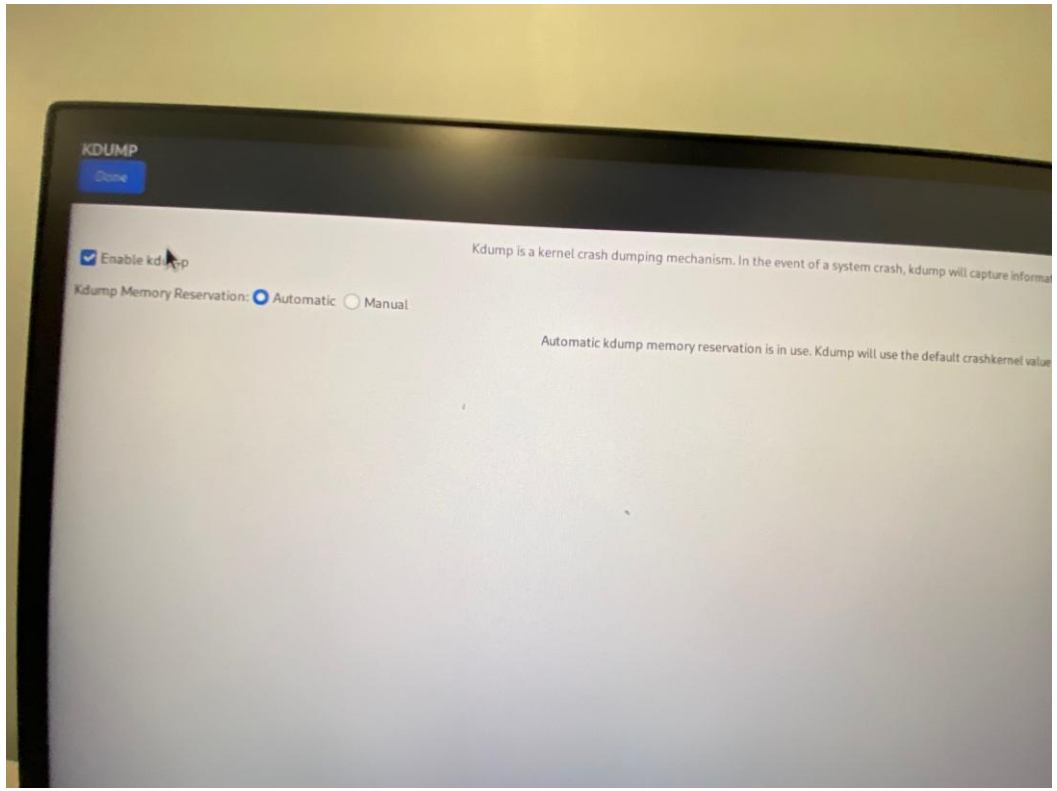


Selected delete all then “reclaim space”

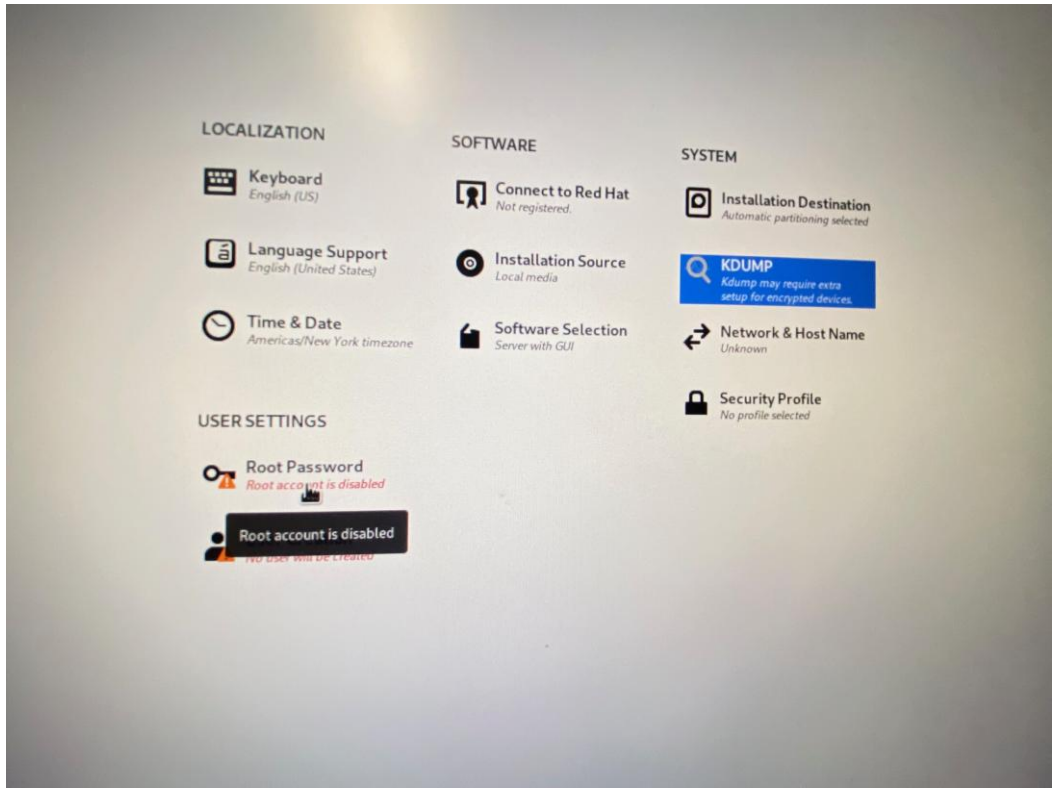


Next click kDump

Approach B: Enable kdump & Select Set root password

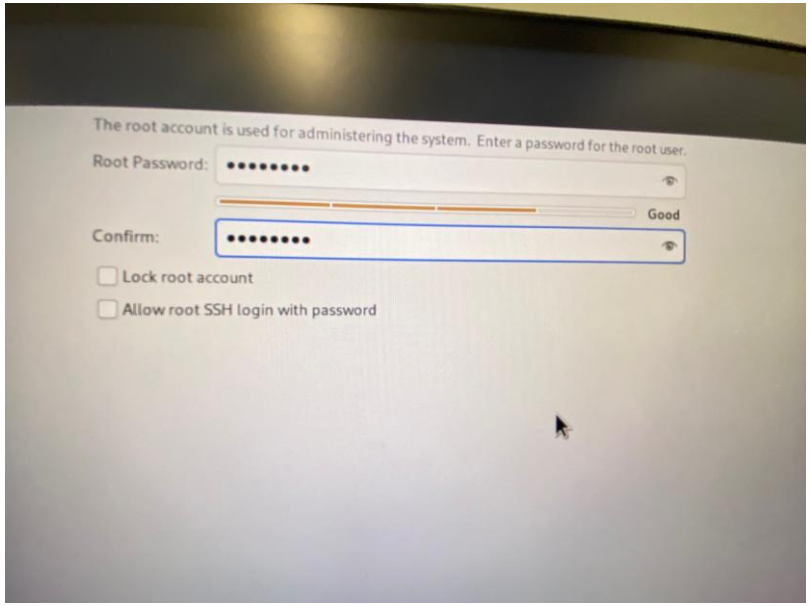


Check Enable kdump -> Done

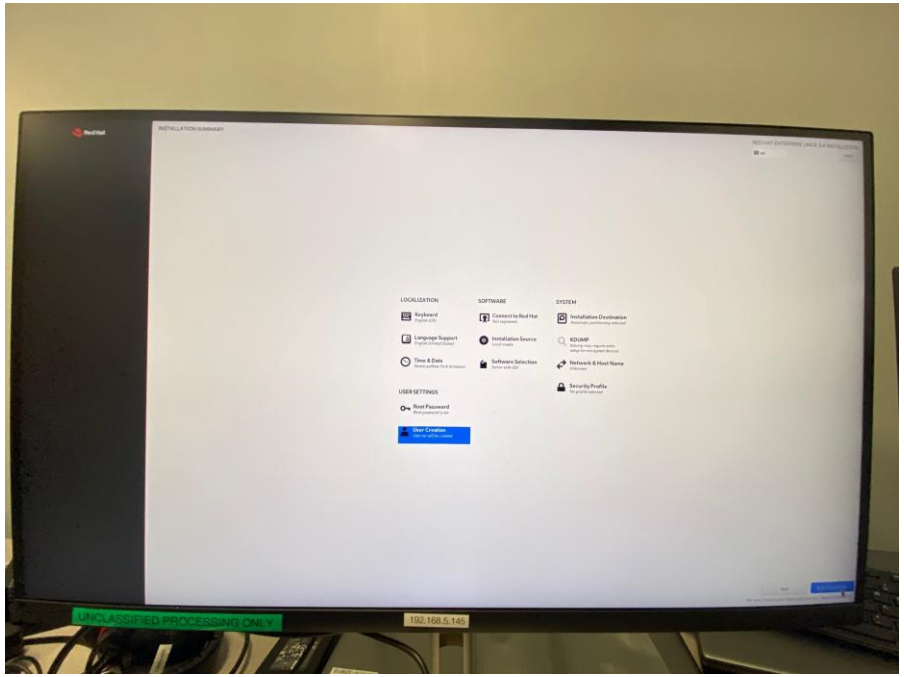


click root password

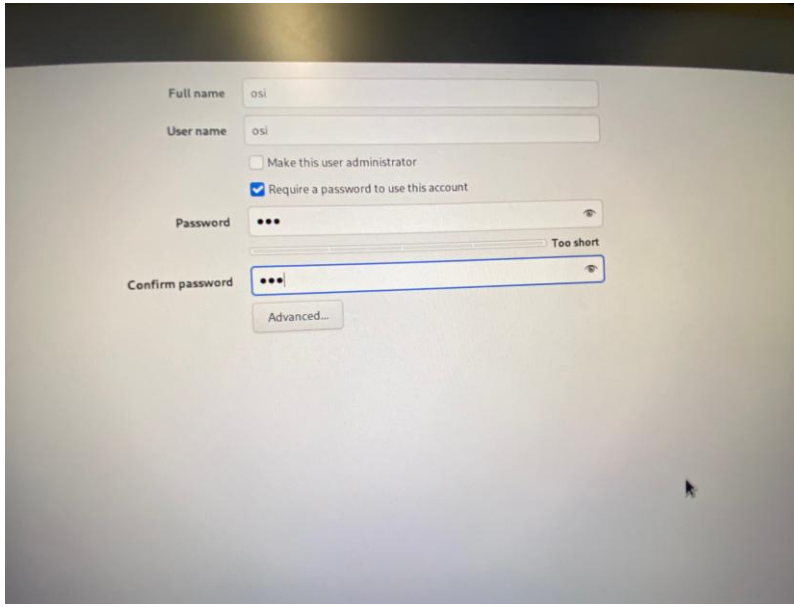
Approach B: set up root and user osi



Input our favorite root password

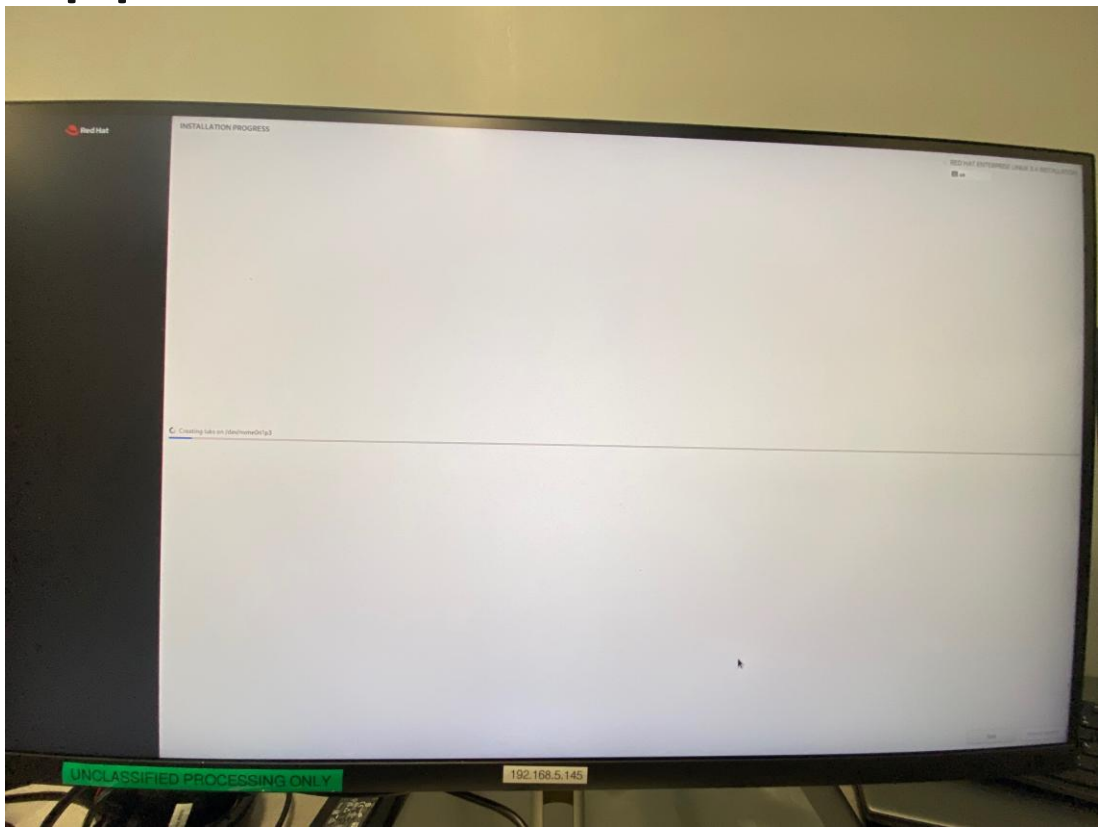


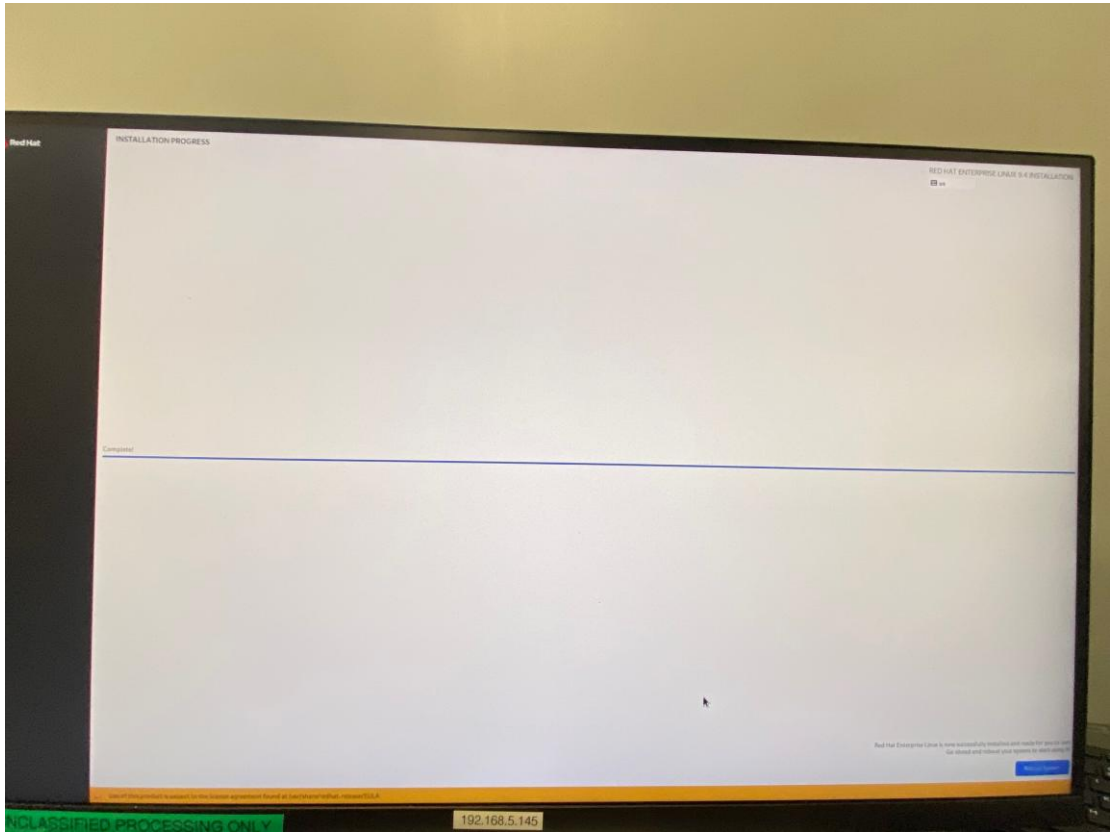
Selecte create user



set osi password, since it’s too short, you need to click “Done” twice

Approach B: Wait Until it’s done





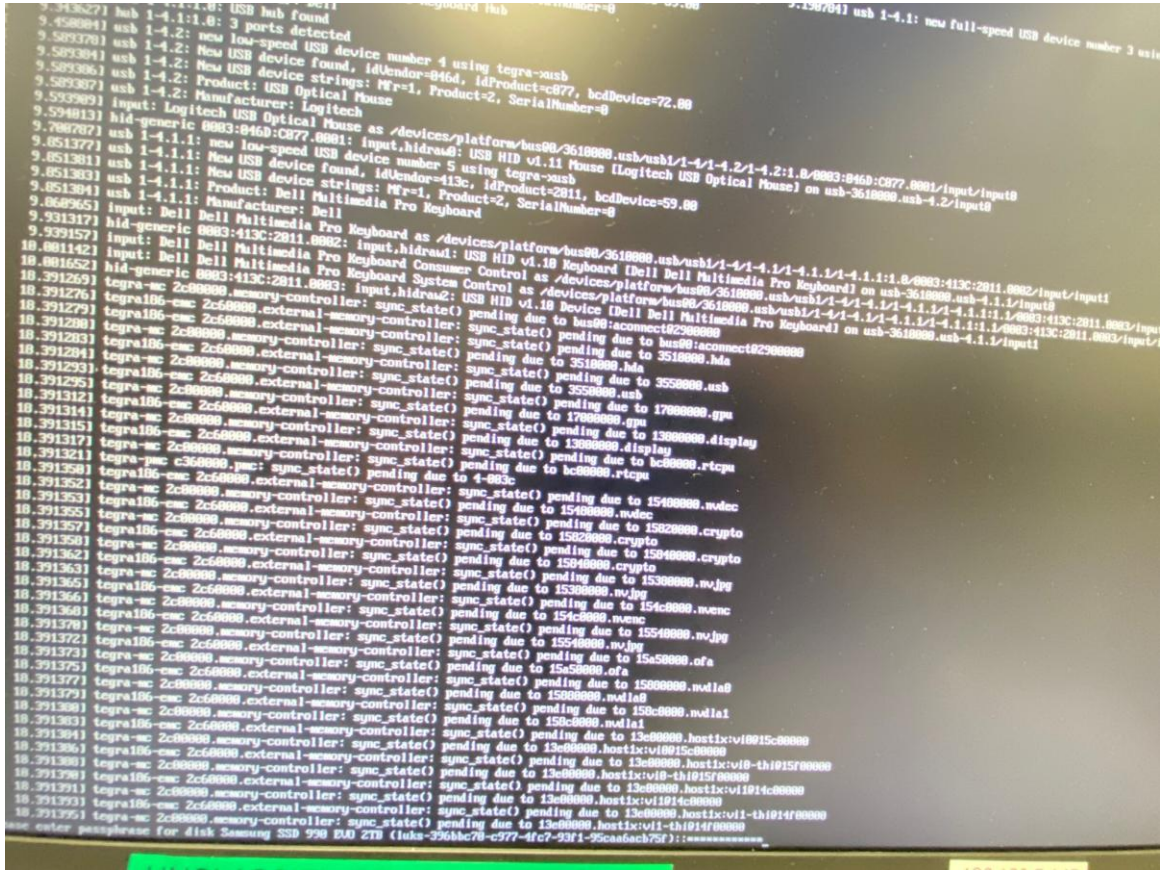
It is install iso, just wait

at the end “reboot” button will be active

click “reboot”, then power off the system from desktop interface
At this stage the approach B is done!

Reboot

1. Power on (without USB drive)



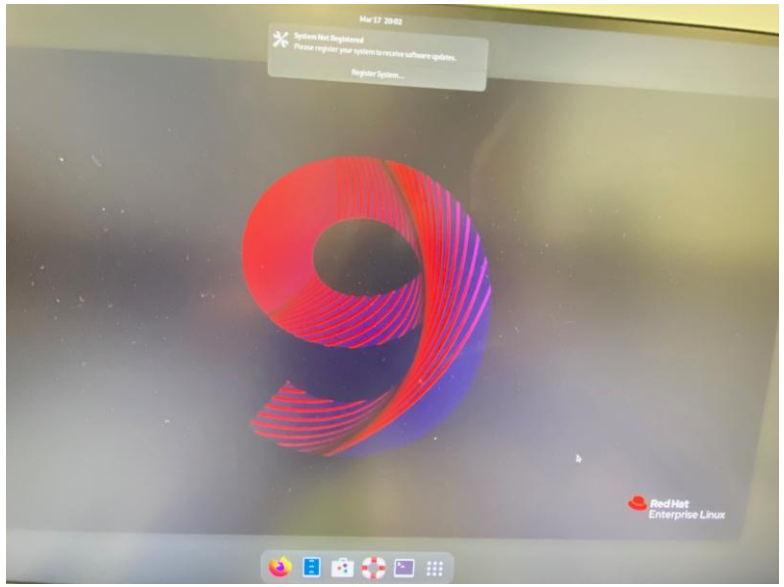
2 Hit any key then input the password of SSD drive (for Approach B only)



3. Log in with our favorite password of osi user

Reboot and verify

1. Reboot and login

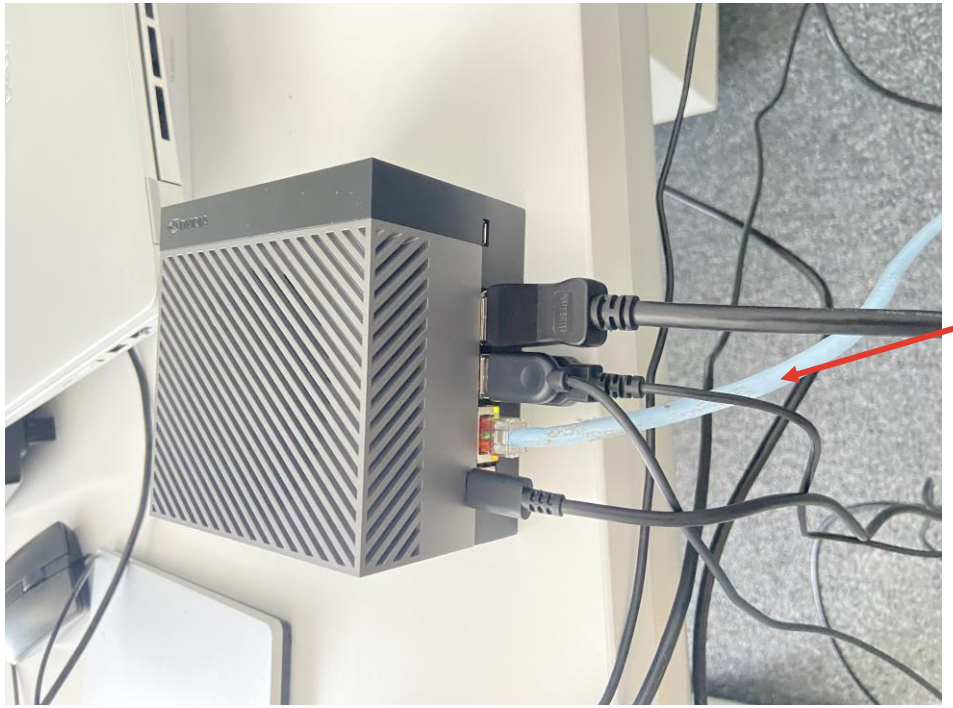


2. Very RHEL is installed on eMMC

```
[osi@localhost ~]$ findmnt /
TARGET SOURCE                FSTYPE OPTIONS
/      /dev/mapper/rhel-root xfs    rw,relatime,seclabel,attr2,inode64,logbufs=8,logbsize=32k,noquota
[osi@localhost ~]$
[osi@localhost ~]$ lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINTS
NAME           SIZE TYPE FSTYPE      MOUNTPOINTS
mmcblk0       59.2G disk
├─mmcblk0p1    600M part vfat        /boot/efi
├─mmcblk0p2      1G part xfs         /boot
└─mmcblk0p3   57.6G part LVM2_member
  ├─rhel-root 34.7G lvm  xfs         /
  ├─rhel-swap  5.9G lvm  swap        [SWAP]
  └─rhel-home   17G lvm  xfs         /home
mmcblk0boot0     4M disk
mmcblk0boot1     4M disk
nvme0n1       1.8T disk
├─nvme0n1p1    600M part vfat
├─nvme0n1p2      1G part xfs
└─nvme0n1p3    1.8T part crypto_LUKS
[osi@localhost ~]$
[osi@localhost ~]$
[osi@localhost ~]$ sudo pvs
  PV             VG   Fmt  Attr PSize   PFree
  /dev/mmcblk0p3 rhel lvm2 a--  <57.63g    0
[osi@localhost ~]$
```

But system is not registered yet!
(to do it we need internet)

Part 3 Set up NVIDIA Kernel Module and Utilities



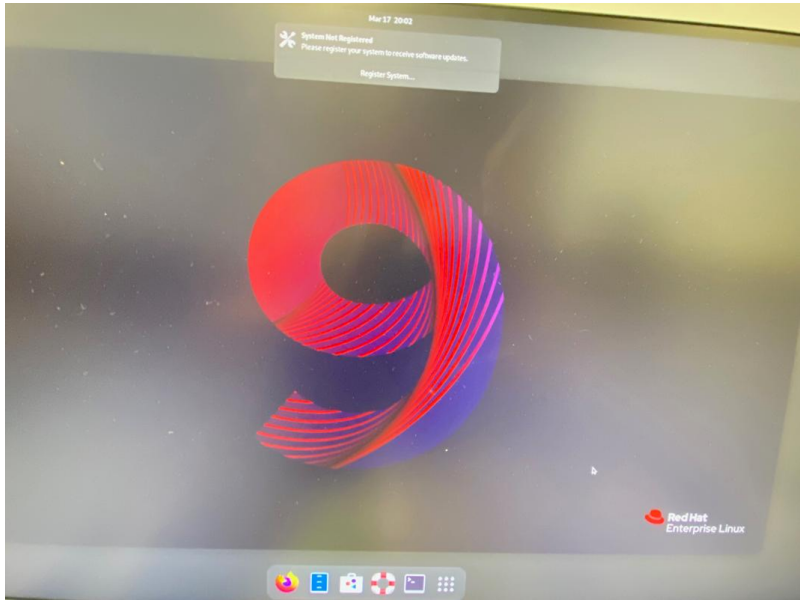
Orin connects with internet via DMZ

Set date time and enable network

1. Reboot and login

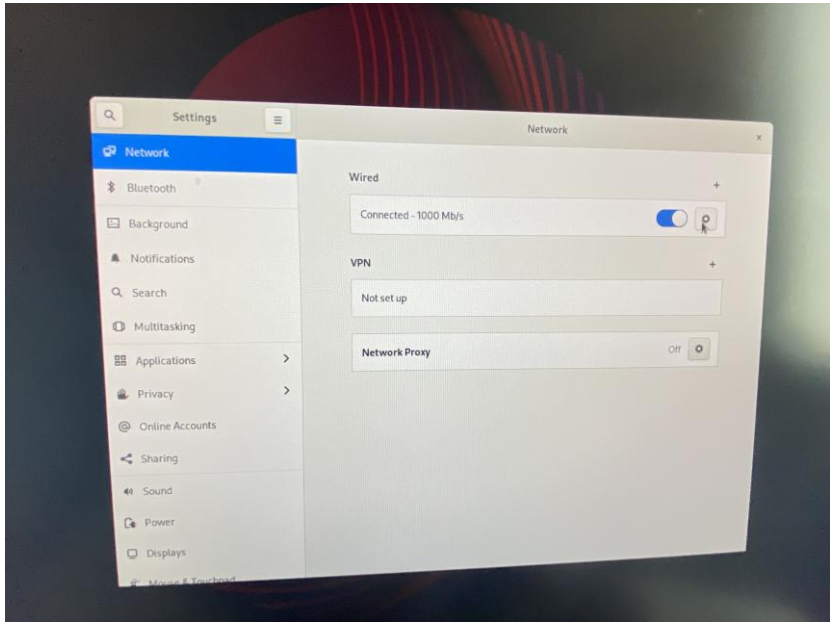
2. Set date and time

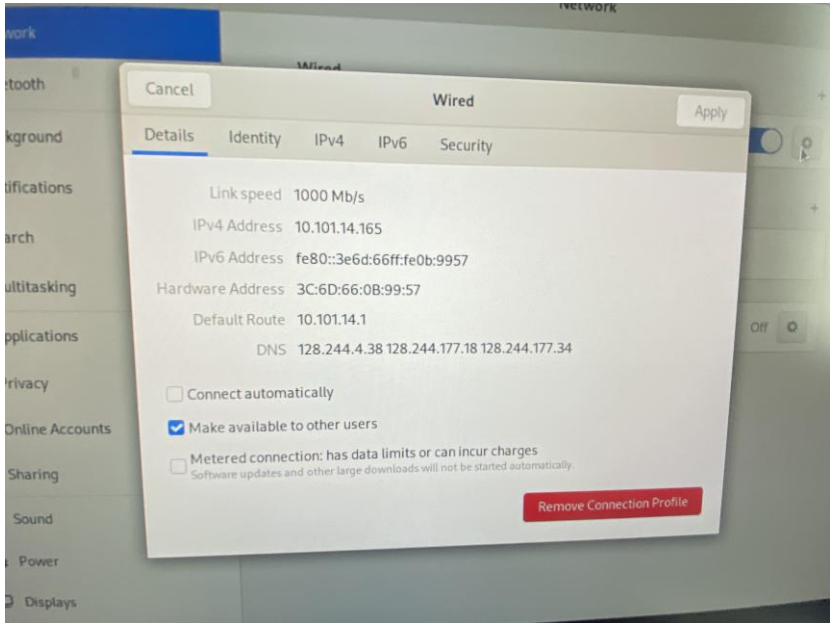
```
$su –
$timedatectl list-timezones
$timedatectl set-timezone America/New_York
$timedatectl set-ntp false
$timedatectl set-time "2026-07-10 19:00:00“
$date
```



3. Set network

Settings -> network -> click wired





Register system

1. Register system

```
$su –
$subscription-manager register
$username: shunguang.wu  (or any APL RHEL registered user)
#password: Maven@JhuApl2026
```

2. Setup sudo for osi

```
$su -
$usermod –aG video osi
$usermod –aG wheel osi
$id osi
```

3. Logout from current osi user, then login again



Set up NVIDIA kernel module and utilities

1. Restart Orin and login as osi user

2 Add NVIDIA repositories:

```
$ curl -s -L https://nvidia.github.io/libnvidia-container/stable/rpm/nvidia-container-toolkit.repo | sudo tee /etc/yum.repos.d/nvidia-container-toolkit.repo
$ curl -s -L https://repo.download.nvidia.com/jetson/rhel-9.4/jp6.1/nvidia-l4t.repo | sudo tee /etc/yum.repos.d/nvidia-l4t.repo
```

(note, as of to date, the current jp6.1 matched with R36.4.0)

3. Configure user groups:

```
$ echo $USER
  osi
```

(important!!! you must be osi, not root)

```
$ sudo usermod -a -G video $USER
$ sudo usermod -a -G video gdm
```

4. Install the NVIDIA kernel modules and utilities:

```
$ sudo dnf -y install nvidia-jetpack-kmod nvidia-jetpack-all nvidia-container-toolkit
```

5. (Optional)

```
$ systemctl get-default
  you will see ”graphical.target” or “multi-user.target”
  if it shows “graphical.target”, skip the next step, otherwise run it.
$ sudo systemctl set-default graphical.target
```

6. Close all the active windows and Restart the system to ensure all the newly installed kernel modules get loaded

After login as osi user, if you are still in the RHEL GUI, which means we are successfully set up the Nidia kernel module and utilities!
Then open a terminal window run:

```
7. sudo nvidia-ctk cdi generate --output=/etc/cdi/nvidia
```

Turn off auto-update

How to Turn Off Automatic Updates on RHEL 9

1. Check for automatic-update services
$systemctl list-timers --all | grep -Ei 'packagekit|dnf|update|upgrade'
$rpm -q dnf-automatic
$systemctl status packagekit.service

If dnf-automatic reports:
  package dnf-automatic is not installed
then no action is needed for `dnf-automatic`.

2. Disable PackageKit automatic updates
Stop PackageKit:
$sudo systemctl stop packagekit.service

Prevent it from being started again:
$sudo systemctl mask packagekit.service

Also disable PackageKit offline updates, if those units exist:
$sudo systemctl mask packagekit-offline-update.service 2>/dev/null
$sudo systemctl mask packagekit-offline-update.timer 2>/dev/null

3. Disable automatic DNF metadata refresh
`dnf-makecache.timer` does **not install updates**, but if you want to eliminate automatic DNF activity completely:
$sudo systemctl disable --now dnf-makecache.timer

4. Verify
$systemctl status packagekit.service
$systemctl is-enabled packagekit.service
$systemctl is-enabled dnf-makecache.timer
$rpm -q dnf-automatic

Expected results should be approximately:
packagekit.service masked / inactive
dnf-makecache.timer disabled
dnf-automatic not installed

Check remaining timers:
$systemctl list-timers --all | grep -Ei 'packagekit|dnf|update|upgrade'
Seeing `mlocate-updatedb.timer` is fine; it is unrelated to OS/package updates.

5. Update RHEL manually when you choose

Check available updates:
$sudo dnf check-update

Install them:
$sudo dnf update

Encrypt and mount NVME (if Choose Approach A)

```
$ lsblk –f
$ findmnt | grep nvme
$ sudo pvs


$sudo cryptsetup luksFormat  /dev/nvme0n1p1
  Are you sure? (Type 'yes' in capital letters): YES

$sudo cryptsetup open /dev/nvme0n1p1 crypt_data
$lsblk –f
$sudo mkfs.xfs /dev/mapper/crypt_data

$sudo mkdir -p /data
$sudo mount /dev/mapper/crypt_data /data
$df -h /data
$findmnt /data
$sudo chown osi:osi /data
$sudo systemctl daemon-reload
$df -h /data
$findmnt /data

$ sudo cat /etc/crypttab
$ cat /etc/fstab
```

```
[osi@localhost ~]$ lsblk -f
NAME          FSTYPE      FSVER    LABEL UUID                                   FSAVAIL
mmcblk0
├─mmcblk0p1   vfat        FAT32          5790-BD16                               591.6M
├─mmcblk0p2   xfs                        325b2b22-6a29-4eef-9de5-cfd795cb2e0a    579.7M
└─mmcblk0p3   LVM2_member LVM2 001       PXkQd9-m5zh-3LNe-psde-yvFL-WecX-v9yDDe
  ├─rhel-root xfs                        6235a6ec-923c-40f7-a055-e00a1b05a122     28.6G
  ├─rhel-swap swap        1              e17c74e5-8726-4564-add2-ace8f5edeb17
  └─rhel-home xfs                        a4663b4e-817f-404a-85c7-29faf029549a     16.6G
mmcblk0boot0
mmcblk0boot1
nvme0n1
└─nvme0n1p1   vfat        FAT32          633C-DA46
```

```
[osi@localhost ~]$ lsblk -f
NAME          FSTYPE      FSVER    LABEL UUID                                   FSAVAIL
mmcblk0
├─mmcblk0p1   vfat        FAT32          5790-BD16                               591.6M
├─mmcblk0p2   xfs                        325b2b22-6a29-4eef-9de5-cfd795cb2e0a    579.7M
└─mmcblk0p3   LVM2_member LVM2 001       PXkQd9-m5zh-3LNe-psde-yvFL-WecX-v9yDDe
  ├─rhel-root xfs                        6235a6ec-923c-40f7-a055-e00a1b05a122     28.6G
  ├─rhel-swap swap        1              e17c74e5-8726-4564-add2-ace8f5edeb17
  └─rhel-home xfs                        a4663b4e-817f-404a-85c7-29faf029549a     16.5G
mmcblk0boot0
mmcblk0boot1
nvme0n1
└─nvme0n1p1   crypto_LUKS 2              88cb00cc-fceb-414d-8a46-f7aeb41f664e
  └─crypt data
```

Final check:

```
$ sudo systemctl status systemd-cryptsetup@crypt_data.service --no-pager
```

Encrypt and mount NVME (if Choose Approach B)

For Approach B, you do not need to do anything, your nvme is already mounted as / and /home

Part 4 Tests and Evaluations

Setup containers folder at /data/podman

```
# step 1: Create the directory and give osi ownership:
$ sudo mkdir -p /data/podman
$ sudo chown -R osi:osi /data/podman

#step 2:  Create the Podman configuration directory:
$ mkdir -p ~/.config/containers

#step 3: Create/edit:
$ vi ~/.config/containers/storage.conf
[storage]
driver = "overlay"
graphroot = "/data/podman"

#Save it, then verify:
$ podman info --format '{{.Store.GraphRoot}}'
It should show:
/data/podman

# step 4: If you don't need the old Podman storage
$ sudo rm -rf /home/osi/.local/share/containers/storage

#Then check disk usage:
df -h /home
df -h /data

#From this point forward, commands such as:
podman pull nvcr.io/nvidia/l4t-jetpack:r36.4.0
podman build -t myimage .
podman run ...

#will store Podman images, container layers, and writable container data under:
/data/podman
```

- Steps 1-4 are for approach A only

or

- Skip steps 1-4 if you choose approach B

Install jtop

```
# Install pip (if needed)
sudo dnf install -y python3-pip

# Install jetson-stats system-wide
sudo -H python3 -m pip install --no-cache-dir -U jetson-stats

# Verify jtop installation
/usr/local/bin/jtop --version

# If the PyPI installation fails, uninstall it
sudo python3 -m pip uninstall -y jetson-stats

# Install directly from the upstream GitHub repository
sudo -H python3 -m pip install \
    --no-cache-dir \
    git+https://github.com/rbonghi/jetson_stats.git

# Verify required files were installed
ls -l /etc/profile.d/jtop_env.sh
ls -l /etc/systemd/system/jtop.service

# Restore SELinux contexts
sudo restorecon -v /etc/profile.d/jtop_env.sh
sudo restorecon -v /etc/systemd/system/jtop.service

# Reload systemd
sudo systemctl daemon-reload
```

```
# Reboot
sudo reboot

# After reboot, start jtop
jtop

# Check the service status (if needed)
systemctl status jtop.service
```

Validate Jetson GPU Support

- Commands:

see
validate-the-Jetson-GPU-support.pdf
Or
https://developers.redhat.com/learning/learn:rhel:install-red-hat-device-edge-nvidia-jetson-orin-and-igx-orin/resource/resources:validate-jetson-gpu-support

- Tested & verified
  - ✓ CUDA,
  - ✓ cuDNN,
  - ✓ TensorRT,
  - ✓ DeepStream Triton container,
  - ✓ GStreamer and DeepStream

Build FFMPEG with NVMPI

- Docker file:
  podman/Dockerfile.ffmpeg
- Build and run container
  ./ podman/podman_script.sh
- Build & install h264, h265, and ffmpeg with nvmpi inside container
  see: build-ffmpeg-with-nvmpi-on-orin-2026-08-10-wus1.pdf
- Test (run these script inside the container
  ./podman/test_h264_generate_input_video.sh
  ./podman/test_h264_generate_input_video.sh
  ./podman/test_h264_nvmpi.sh
  ./podman/test_h265_nvmpi.sh

NVMPI enc/dec Test Summary

- JTOP:
  NVENC: 115 → 980 MHz
  NVDEC: 115 → 148 MHz

- **H.264 NVmpi decoding**

FFmpeg successfully used h264_nvmpi to hardware-decode a 1920×1080, 30-fps, 120-second H.264 video on the Jetson AGX Orin. The NVIDIA multimedia decoder was initialized successfully, and the test decoded 3,592 frames at approximately **462 fps (15.4× real time)**, completing in **7.78 seconds**.

- **H.264 NVmpi encoding**

FFmpeg successfully used h264_nvmpi and the Orin **NVENC hardware encoder** to encode the same 1080p30 video at 8 Mbps. It encoded 3,600 frames at approximately **168 fps (5.61× real time)**, completing in **21.37 seconds**. In this test, input decoding was still performed by FFmpeg's native software H.264 decoder, while encoding was hardware-accelerated by NVENC.

- **H.265 NVmpi decoding**

FFmpeg successfully used hevc_nvmpi to hardware-decode the 120-second, 1920×1080, 30-fps H.265/HEVC video. The NVIDIA multimedia decoder was initialized successfully, and all 3,600 frames were decoded at approximately **683 fps (22.8× real time)**, completing in only **5.27 seconds**. CPU time was relatively low (utime=3.07s, stime=4.37s), confirming effective hardware decode offload.

- **H.265 NVmpi encoding**

The hevc_nvmpi encoder successfully used the Orin **NVENC hardware encoder** to produce an 8-Mbps 1080p30 HEVC stream. With software HEVC decoding, encoding achieved **94 fps (3.14× real time)** in **38.18 seconds**. Using hevc_nvmpi for both hardware decoding and encoding maintained essentially the same **94 fps / 3.14×** throughput, but substantially reduced CPU time from about **36.5 CPU-seconds to 13.4 CPU-seconds**, demonstrating the benefit of hardware acceleration on both sides of the transcoding pipeline.

Change IP (assume Orin ip: 192.168.7.71, mask 255.255.252.0)

```
$nmcli connection show
NAME UUID                                  TYPE      DEVICE
APL  xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx  ethernet  enp1s0

$nmcli device status

$nmcli connection modify "APL" \
    ipv4.addresses 192.168.7.71/22 \
    ipv4.method manual

$nmcli connection modify "APL" \
    ipv4.addresses 192.168.7.71/22 \
    ipv4.gateway 192.168.4.1 \
    ipv4.dns "8.8.8.8 1.1.1.1" \
    ipv4.method manual

$nmcli connection down "APL"
$nmcli connection up "APL“

$nmcli connection show "APL"
$ip addr show
$ip route
```